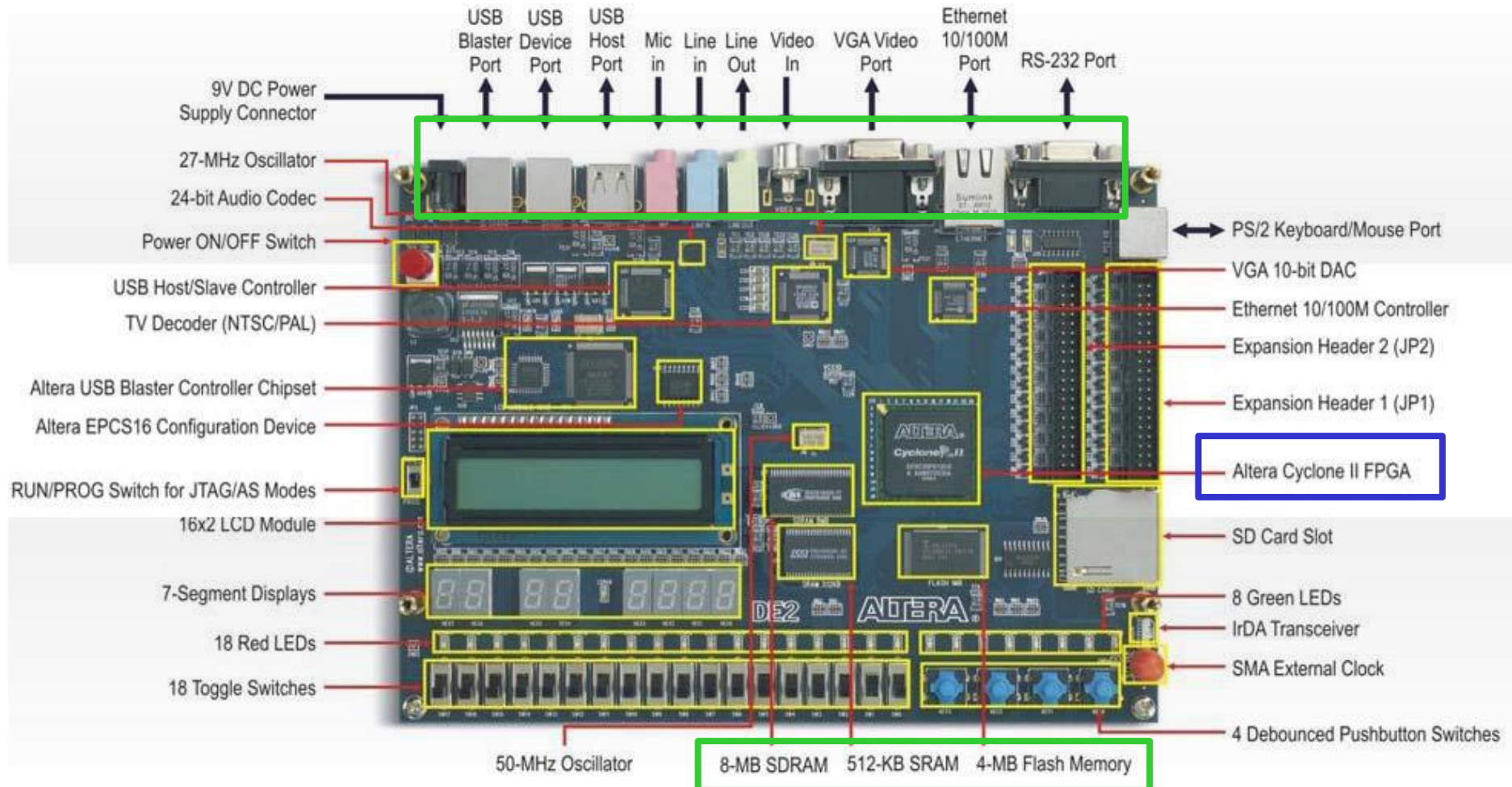
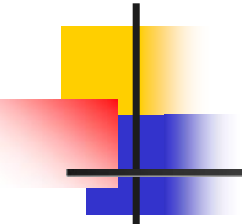


進階模組設計

- ❑ **FPGA 多媒體開發平台(DE2)**
- ❑ **Example: Digital Clock**
- ❑ **IIC (I²C)**

DE2實驗版俯視圖





友晶科技FPGA產品

- ❑ 實驗版為友晶科技有限公司的多媒體開發平台。
- ❑ Altera DE2多媒體開發平台特性：
 - Altera Cyclone II 2C35 晶片
 - 內嵌USB Blaster(爆破工)燒錄
 - AS Mode 及 JTAG Mode 燒錄方式
 - 50 MHz Oscillator and 27MHz Oscillator
 - 24-bit CD-Quality Audio CODEC (語音處理)
 - TV Decoder (NTSC/PAL) (視訊信號處理)
 - **可外接 CAMERA 以及 VGA (訊號處理)**
 - 彈性的使用者規劃接腳 (擴充功能)

DE2實驗版詳細規格 (1)



Signal Name	FPGA Pin No.	Description
SW[0]	PIN_N25	Toggle Switch [0]
SW[1]	PIN_N26	Toggle Switch [1]
SW[2]	PIN_P25	Toggle Switch [2]
SW[3]	PIN_AE14	Toggle Switch [3]
SW[4]	PIN_AF14	Toggle Switch [4]
SW[5]	PIN_AD13	Toggle Switch [5]
SW[6]	PIN_AC13	Toggle Switch [6]
SW[7]	PIN_C13	Toggle Switch [7]
SW[8]	PIN_B13	Toggle Switch [8]
SW[9]	PIN_A13	Toggle Switch [9]
SW[10]	PIN_N1	Toggle Switch [10]
SW[11]	PIN_P1	Toggle Switch [11]
SW[12]	PIN_P2	Toggle Switch [12]
SW[13]	PIN_T7	Toggle Switch [13]
SW[14]	PIN_U3	Toggle Switch [14]
SW[15]	PIN_U4	Toggle Switch [15]
SW[16]	PIN_V1	Toggle Switch [16]
SW[17]	PIN_V2	Toggle Switch [17]

DE2實驗版詳細規格 (2)



Signal Name	FPGA Pin No.	Description
KEY[0]	PIN_G26	Pushbutton[0]
KEY[1]	PIN_N23	Pushbutton[1]
KEY[2]	PIN_P23	Pushbutton[2]
KEY[3]	PIN_W26	Pushbutton[3]

DE2實驗版詳細規格 (3)

The feature set of the TRDB_LCM is listed below:

1. Equipped with Toppoly TD036THEA1 compact TFT LCD module.
2. Handle digital signals of serial 8-bit (RGB or YUV).
3. Support NTSC and PAL timings.
4. 3-wire register control for display and function selection.
5. Built-in contrast, brightness and gamma modulation.
6. Support strip color filter 960x240(through mode, RGB dummy, YUV input).
7. The general specifications of Panel is listed below:

Item	Description	Unit
Display Size (Diagonal)	3.6	Inch
Display Type	Transmissive	-
Active Area (HxV)	72.96 x 54.72	mm
Number of Dots (HxV)	320 x RGB x 240	dot
Dot Pitch (HxV)	0.076 x 0.228	mm
Color Arrangement	RGB Stripe	-
Color Numbers	8 bit RGB (16M color)	-



DE2實驗版詳細規格 (4)

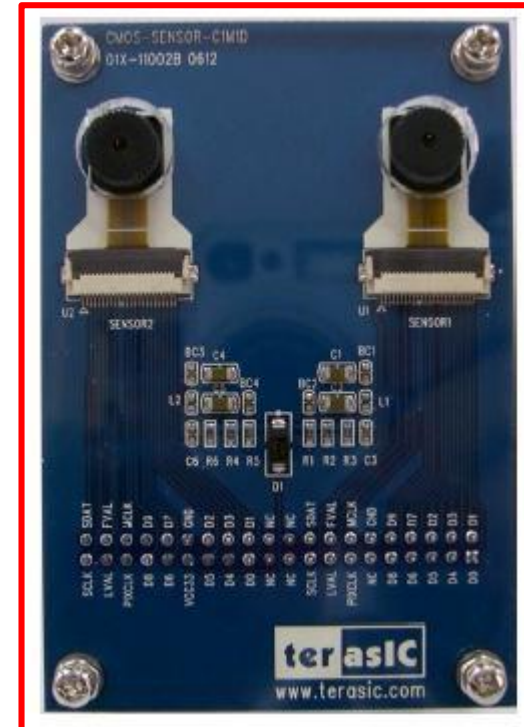
The TRDB_LCM has a 40-pin connector. The pin description of the 40-pin connector follows:

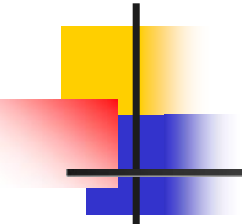
Pin Numbers	Name	Direction	Description
1~10	NC	N/A	Not connect
11	VCC5	N/A	Power 5V
12	GND	N/A	Ground
13~20	NC	N/A	Not connect
21	DIN6	Input	LCD data bus bit 7
22	DIN7	Input	LCD data bus bit 6
23	DIN4	Input	LCD data bus bit 4
24	DIN5	Input	LCD data bus bit 5
25	DIN2	Input	LCD data bus bit 2
26	DIN3	Input	LCD data bus bit 3
27	DIN0	Input	LCD data bus bit 0
28	DIN1	Input	LCD data bus bit 1
29	VCC33	N/A	Power 3.3V
30	NC	N/A	Not connect
31	VSYNC	Input	Vertical sync input
32	NC	N/A	Not connect
33	SCL	Input	3-wire serial interface clock
34	DCLK	Input	LCD data clock
35	GRESTB	Input	Global reset, low active
36	SHDB	Input	Shutdown control, low active
37	CPW	N/A	Reserved
38	SCEN	Input	3-wire serial interface enable
39	SDA	Input/Output	3-wire serial interface data
40	HSYNC	Input	Horizontal sync input

DE2實驗版詳細規格 (5)

The TRDB_DC2 has a 40-pin connector on the back of the board. The pin description of the 40-pin connector follows:

Pin Numbers	Name	Direction	Description
1	DATA1[0]	Output	Sensor 1 Data Bit 0
2	DATA1[1]	Output	Sensor 1 Data Bit 1
3	DATA1[4]	Output	Sensor 1 Data Bit 4
4	DATA1[3]	Output	Sensor 1 Data Bit 3
5	DATA1[5]	Output	Sensor 1 Data Bit 5
6	DATA1[2]	Output	Sensor 1 Data Bit 2
7	DATA1[6]	Output	Sensor 1 Data Bit 6
8	DATA1[7]	Output	Sensor 1 Data Bit 7
9	DATA1[8]	Output	Sensor 1 Data Bit 8
10	DATA1[9]	Output	Sensor 1 Data Bit 9
11	NC	N/A	Not Connect
12	GND	N/A	Ground
13	PIXCLK1	Output	Sensor 1 Pixel Clock
14	MCLK1	Input	Sensor 1 Master Clock
15	LVAL1	Output	Sensor 1 Line Valid
16	FVAL1	Output	Sensor 1 Frame Valid
17	SCLK1	input	Sensor 1 I ² C Clock
18	SDATA1	I/O	Sensor 1 I ² C Data
19	N/C	N/A	Not Connect
20	N/C	N/A	Not Connect



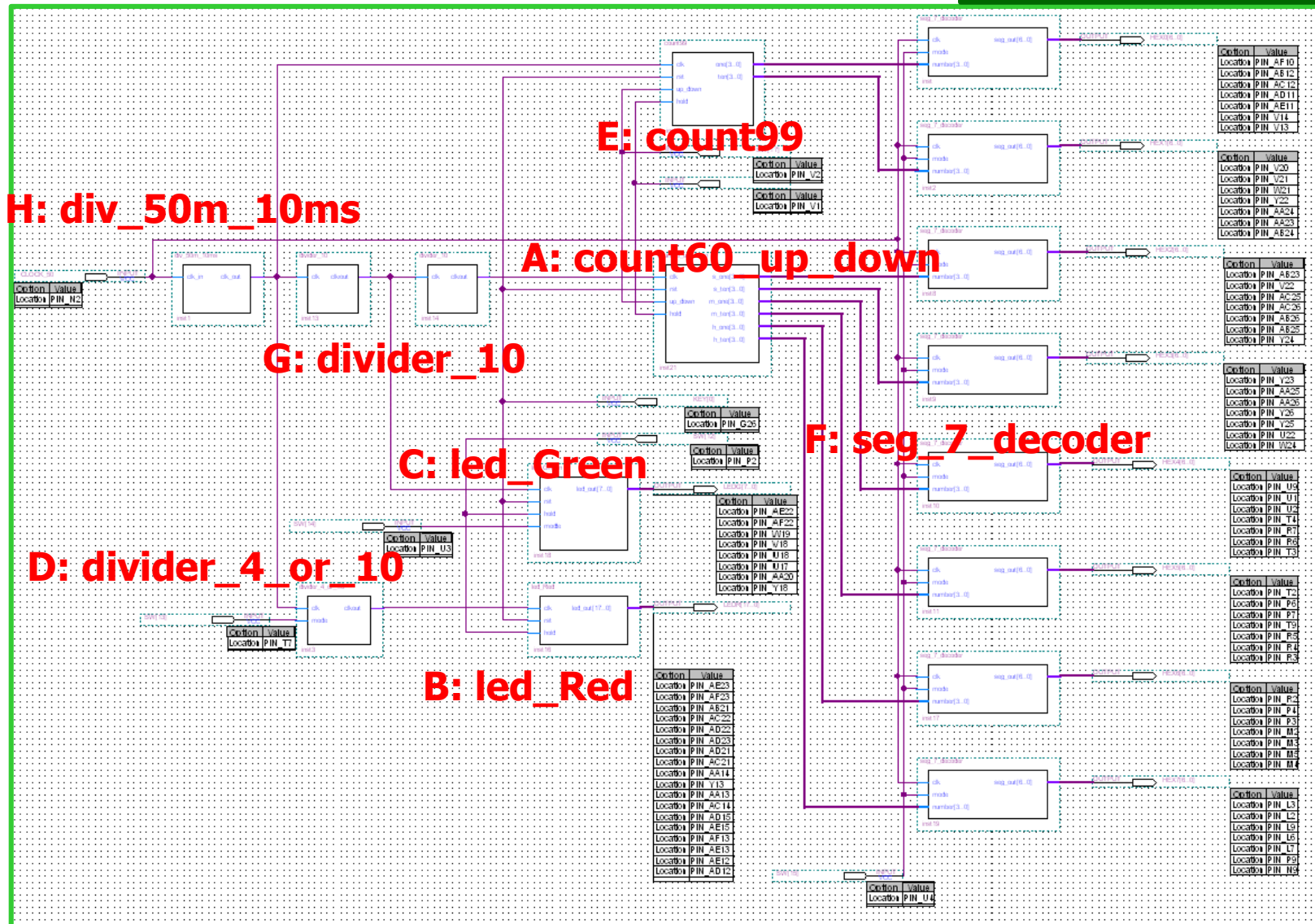


進階模組設計

- ❑ FPGA 多媒體開發平台(DE2)
- ❑ **Example: Digital Clock**
- ❑ IIC (I²C)

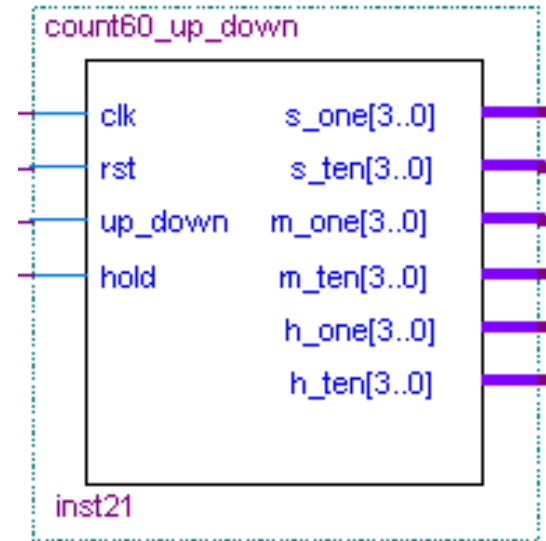
System Architecture for Digital Clock

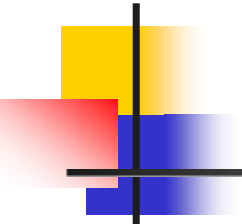
6_00_DE2_Digital clock



Appendix A: count60_up_down (1/2)

```
1. if (up_down==1) // up counter
2. begin // 1
3.   if(s_one_temp==4'd9)
4.     begin // 2
5.       s_one_temp=4'd0;
6.       if(s_ten_temp==4'd5)
7.         begin // 3
8.           s_ten_temp=4'd0;
9.           if(m_one_temp==4'd9)
10.            begin // 4
11.              m_one_temp=4'd0;
12.              if(m_ten_temp==4'd5)
13.                begin // 5
14.                  m_ten_temp=4'd0;
15.                  if((h_ten_temp==4'd2)&&(h_one_temp==4'd3))
16.                    begin // 6
17.                      h_one_temp=4'd0;
18.                      h_ten_temp=4'd0;
19.                    end // 6
20.                  else
21.                    begin // 7
22.                      .....
```





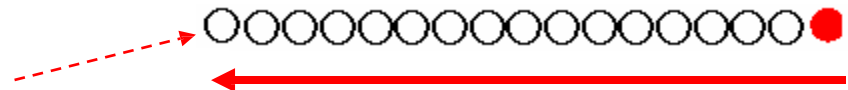
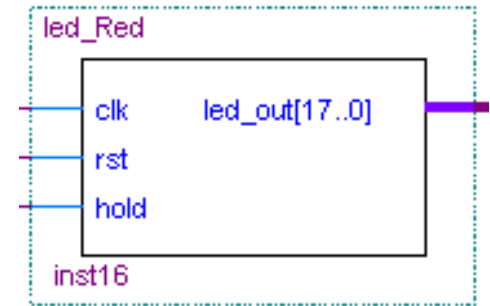
Appendix A: count60_up_down (2/2)

```
70. ....
71.else // down counter
72. begin
73. begin // 1
74.   if(s_one_temp==4'd0)
75.     begin // 2
76.       s_one_temp=4'd9;
77.       if(s_ten_temp==4'd0)
78.         begin // 3
79.           s_ten_temp=4'd5;
80.           if(m_one_temp==4'd0)
81.             begin // 4
82.               m_one_temp=4'd9;
83.               if(m_ten_temp==4'd0)
84.                 begin // 5
85.m_ten_temp=4'd5; 86.

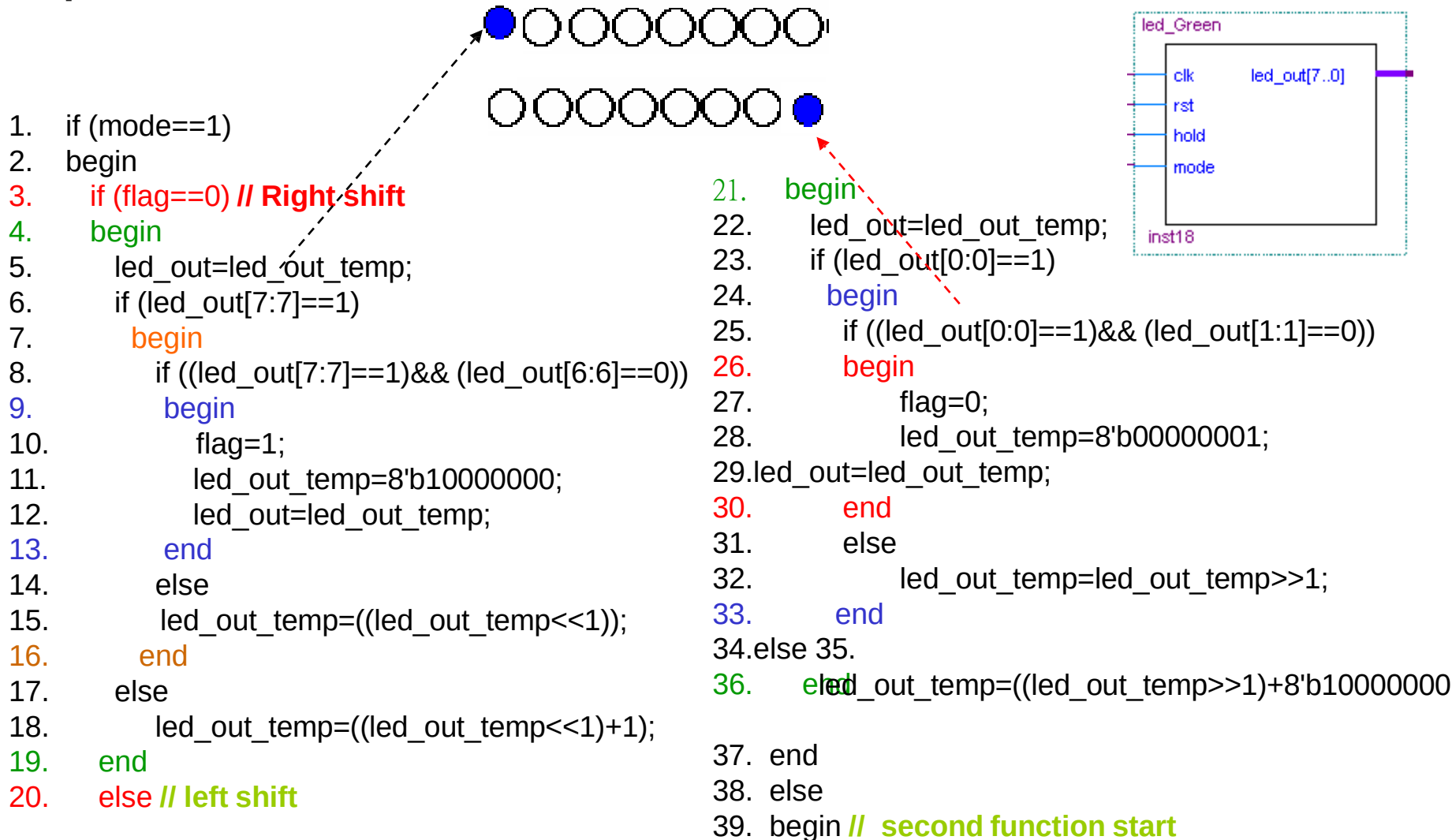
           if((h_ten_temp==4'd0)&&(h_one_temp==4'd
0))
87.           begin // 6
88.             h_one_temp=4'd3;
89.             h_ten_temp=4'd2;
90.           end // 6
91.           ....
```

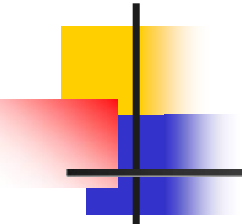
Appendix B: led_Red

```
1.  if (flag==1) // Right shift
2.      begin
3.          led_out=led_out_temp;
4.          if (led_out[17:17]==1)
5.              begin
6.                  flag=0;
7.                  led_out_temp=led_out_temp>>1;
8.              end
9.          else
10.             led_out_temp=led_out_temp<<1;
11.        end
12.    else // left shift
13.        begin
14.            led_out=led_out_temp;
15.            if (led_out[0:0]==1)
16.                begin
17.                    flag=1;
18.                    led_out_temp=led_out_temp<<1;
19.                end
20.            else
21.                led_out_temp=led_out_temp>>1;
22.        end
```



Appendix C: led_Green (1/2)





Appendix C: led_Green (2/2)

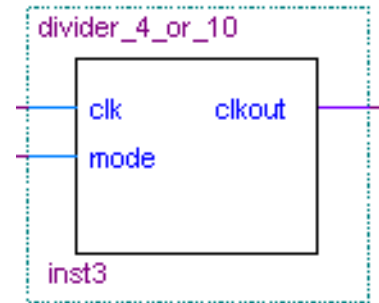
```
40. if (flag==0) // Right shift
41. begin
42.     led_out=led_out_temp;
43.     if (led_out[7:7]==0)
44.         begin
45.             if ((led_out[7:7]==0)&& (led_out[6:6]==1))
46.                 begin
47.                     flag=1;
48.                     led_out_temp=8'b01111111;
49.                     led_out=led_out_temp;
50.                 end
51.             else
52.                 led_out_temp=((led_out_temp<<1));
53.         end
54.     else
55.         led_out_temp=((led_out_temp<<1)+1);
56. end

57. else // left shift
58. begin
59.     led_out=led_out_temp;
60.     if (led_out[0:0]==0)
61.         begin
62.             if ((led_out[0:0]==0)&& (led_out[1:1]==1))
63.                 begin
64.                     flag=0;
65.                     led_out_temp=8'b11111110;
66.                     led_out=led_out_temp;
67.                 end
68.             else
69.                 led_out_temp=led_out_temp>>1;
70.         end
71.     else
72.         led_out_temp=((led_out_temp>>1)
                        + 8'b10000000) ;
73. end
74. end // second function end
```


Appendix D: divider_4_or_10

```
1. always @(posedge clk )
2. begin
3.     if (mode==1)
4.         begin
5.             if (Qtemp == 3)
6.                 Qtemp = 0;
7.             else
8.                 Qtemp = Qtemp + 1;
9.         end
10.    else
11.        begin
12.            if (Qtemp == 9)
13.                Qtemp = 0;
14.            else
15.                Qtemp = Qtemp + 1;
16.        end
17. end
```

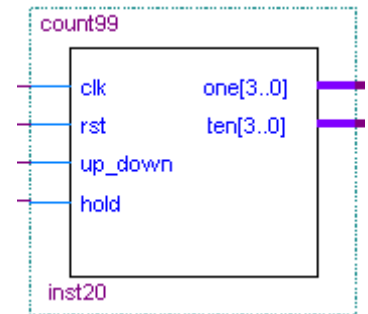
```
18. always @(Qtemp)
19. begin
20.     if (mode==1)
21.         begin
22.             if (Qtemp == 3)
23.                 clkout = 1;
24.             else
25.                 clkout = 0;
26.         end
27.    else
28.        begin
29.            if (Qtemp == 9)
30.                clkout = 1;
31.            else
32.                clkout = 0;
33.        end
34.
35. end
```



Appendix E: count99

```
1. if (hold == 0)
2.   begin
3.     if (up_down==1) // up counter
4.       begin // 1
5.         if(one_temp==4'd9)
6.           begin // 2
7.             one_temp=4'd0;
8.             if(ten_temp==4'd9)
9.               begin // 3
10.                ten_temp=4'd0;
11.              end
12.            else
13.              begin // 12
14.                ten_temp=ten_temp+4'd1;
15.              end // 12
16.            end // 2
17.          else
18.            begin // 13
19.              one_temp=one_temp+4'd1;
20.            end // 13
21.            one=one_temp;
22.            ten=ten_temp;
23.          end // 1
24.        end
```

```
25. else // down counter
26.   begin // 1
27.     if(one_temp==4'd0)
28.       begin // 2
29.         one_temp=4'd9;
30.         if(ten_temp==4'd0)
31.           begin // 3
32.             ten_temp=4'd9;
33.           end // 3
34.         else
35.           begin // 12
36.             ten_temp=ten_temp-4'd1;
37.           end // 12
38.         end // 2
39.       else
40.         begin // 13
41.           one_temp=one_temp-4'd1;
42.         end // 13
43.         one=one_temp;
44.         ten=ten_temp;
45.       end // 1
```

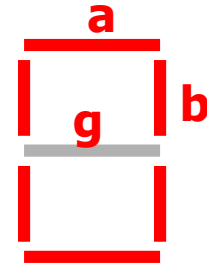
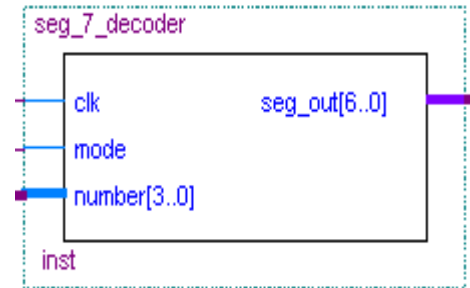


Appendix F: seg_7_decoder

```

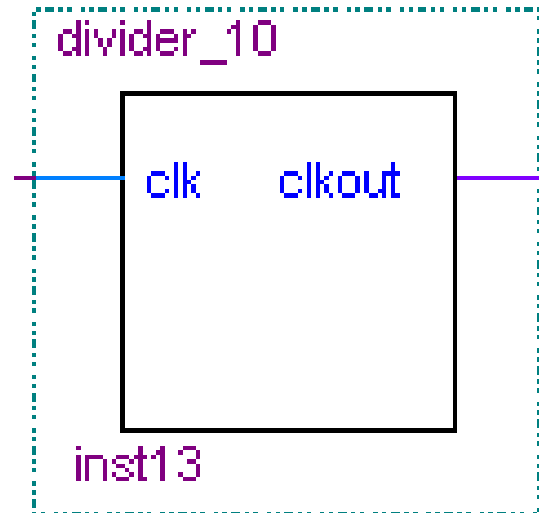
1. case(number)
2.     4'd0:
3.         begin
4.             if (mode==1)
5.                 begin
6.                     seg_out=7'b1000000; // Condition 1
7.                 end
8.             else
9.                 begin
10.                     seg_out=7'b0111111; // Condition 2
11.                 end
12.             end
13.     4'd1:
14.         begin
15.             if (mode==1)
16.                 begin
17.                     seg_out=7'b1111001; // Condition 1
18.                 end
19.             else
20.                 begin
21.                     seg_out=7'b0000110; // Condition 2
22.                 end
23.             end
24.         .....

```



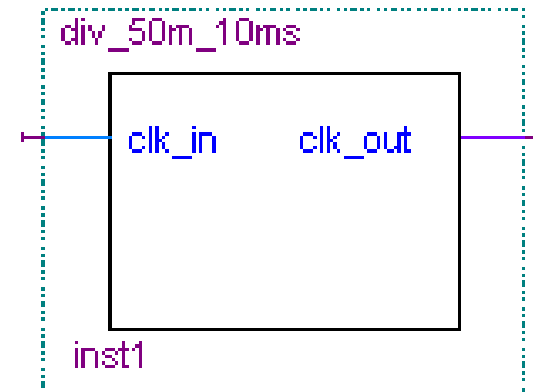
Appendix G: divider_10

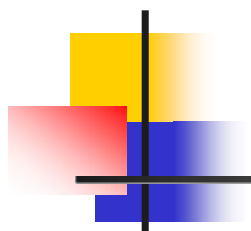
```
1. module divider_10 (clk, clkout);
2.     input clk;
3.     output clkout;
4.     reg clkout;
5.     reg [6:0] Qtemp;
6.     always @(posedge clk )
7.         begin
8.             if (Qtemp == 9)
9.                 Qtemp = 0;
10.            else
11.                Qtemp = Qtemp + 1;
12.        end
13.     always @(Qtemp)
14.         begin
15.             if (Qtemp == 9)
16.                 clkout = 1;
17.             else
18.                 clkout = 0;
19.         end
20. endmodule
```



Appendix H: div_50m_10ms

```
1. module div_50m_10ms(clk_in,clk_out);
2.     input clk_in;
3.     output clk_out;
4.     reg clk_out;
5.     reg [18:0]count; // need to re-modify
6.     always@(posedge clk_in)
7.         begin
8.             if(count==19'd500000) // need to re-modify
9.                 begin
10.                    count=0;
11.                end
12.            else
13.                count=count+1;
14.        end
15.        always@(count)
16.            begin
17.                clk_out=(count==19'd500000)?1'd1:1'd0;
18.            end
19. endmodule
```





Pins Assigned with Excel File

Pins Assigned with Excel File (1)

The screenshot displays the Quartus II software interface. The 'Assignments' menu is open, showing options like 'Device...', 'Pins', 'Timing Analysis Settings...', 'EDA Tool Settings...', 'Settings...', 'Classic Timing Analyzer Wizard...', 'Assignment Editor', 'Pin Planner', 'Remove Assignments...', 'Demote Assignments...', 'Back-Annotate Assignments...', 'Import Assignments...', 'Export Assignments...', 'Assignment (Time) Groups...', 'Timing Closure Floorplan', 'LogicLock Regions Window', and 'Design Partitions Window'. The 'Pins' option is highlighted. The main workspace shows a circuit diagram with components like 'div_50m_10ms', 'divider_10', and 'divider_10'. A message window at the bottom displays system messages, including 'Info: Import completed. 0 assignments were written (out of 0 read). 0 non-global assignments were skipped because of' and 'Warning: Can't'. A red box highlights the text '工具列→Assignments→Pins' (Toolbox→Assignments→Pins) in the message window.

工具列→Assignments→Pins

Opens the Pin Planner

Pins Assigned with Excel File (2)

Quartus II - D:/研究所/研究所雜物/chi chung/Quartus/Qtus_II_codes/CH05_Nios_codes/CH05_01_DE2_Digital clock_codes/seg_7_main(bottom...

File Edit View Processing Tools Window

Top View - Wire Bond
Cyclone II - EP2C35F672C8

加入前的
腳位圖

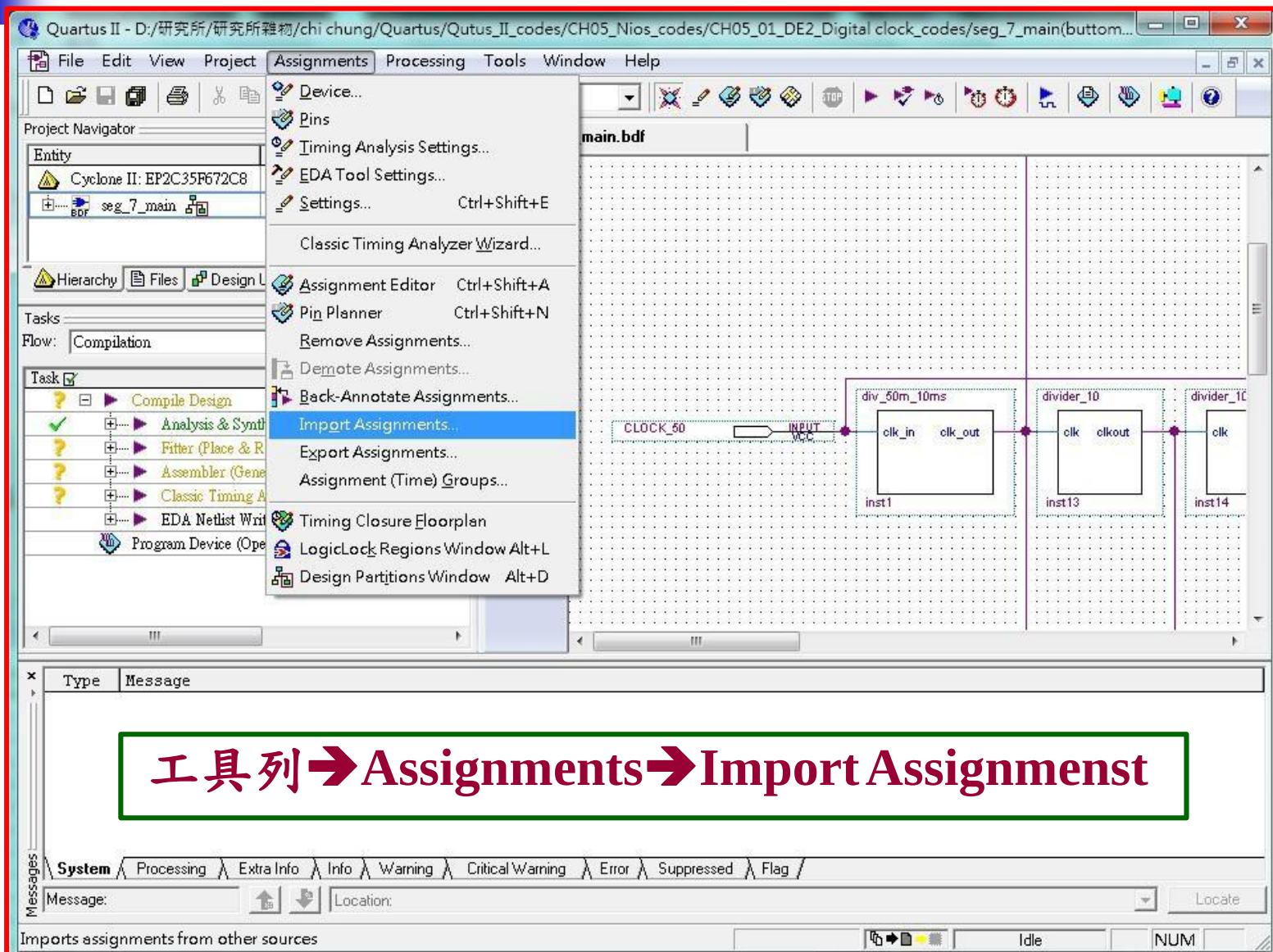
Named: PIN_U17 Filter: Pins: all

	Node Name	Direction	Location	I/O Bank	VREF Group	I/O Standard	Reserved
1	CLOCK_50	Input				3.3-V LVTTTL (default)	
2	HEX0[6]	Output				3.3-V LVTTTL (default)	
3	HEX0[5]	Output				3.3-V LVTTTL (default)	
4	HEX0[4]	Output				3.3-V LVTTTL (default)	
5	HEX0[3]	Output				3.3-V LVTTTL (default)	
6	HEX0[2]	Output				3.3-V LVTTTL (default)	
7	HEX0[1]	Output				3.3-V LVTTTL (default)	
8	HEX0[0]	Output				3.3-V LVTTTL (default)	
9	HEX1[6]	Output				3.3-V LVTTTL (default)	
10	HEX1[5]	Output				3.3-V LVTTTL (default)	
11	HEX1[4]	Output				3.3-V LVTTTL (default)	
12	HEX1[3]	Output				3.3-V LVTTTL (default)	

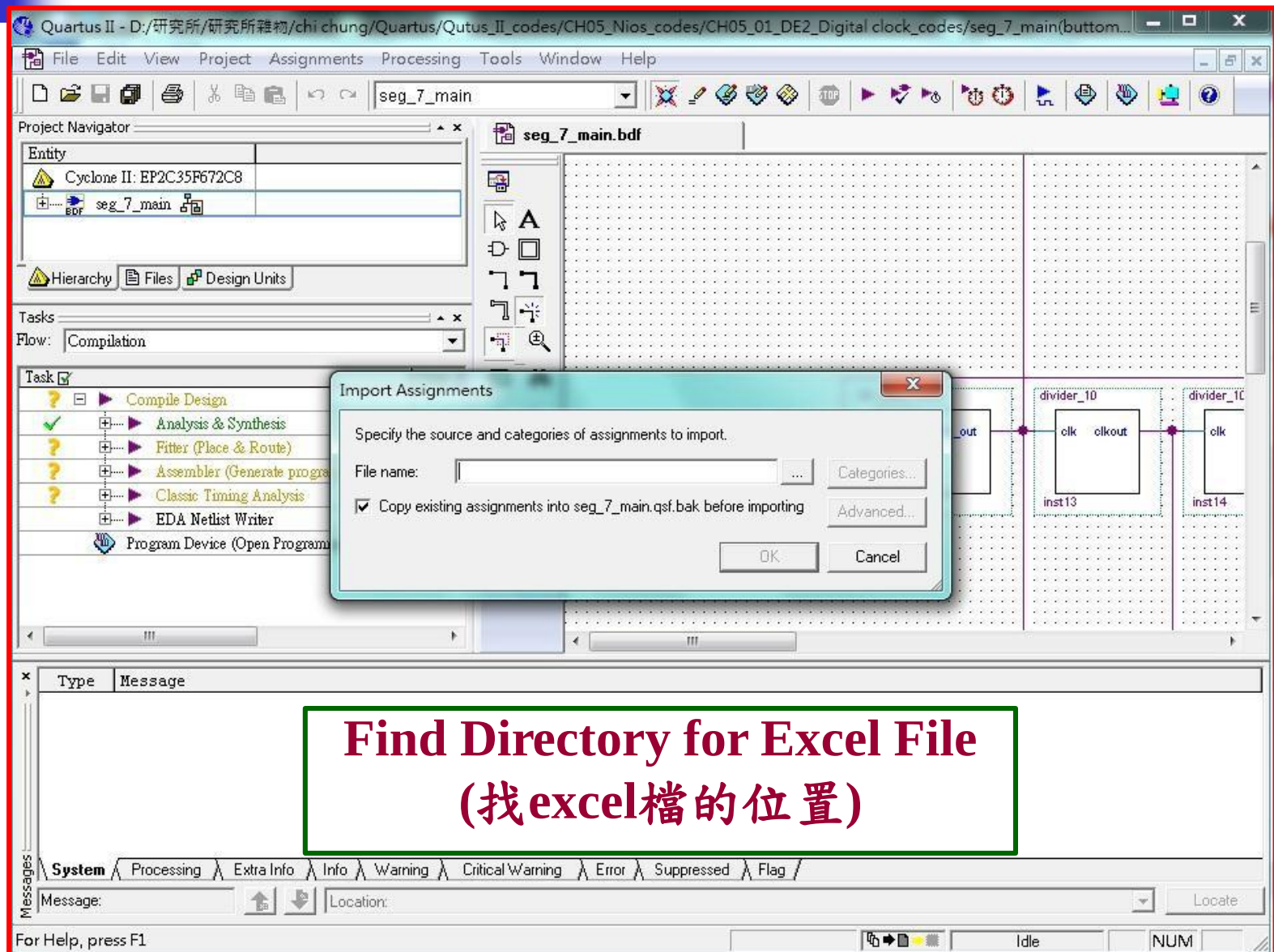
For Help, press F1

NUM

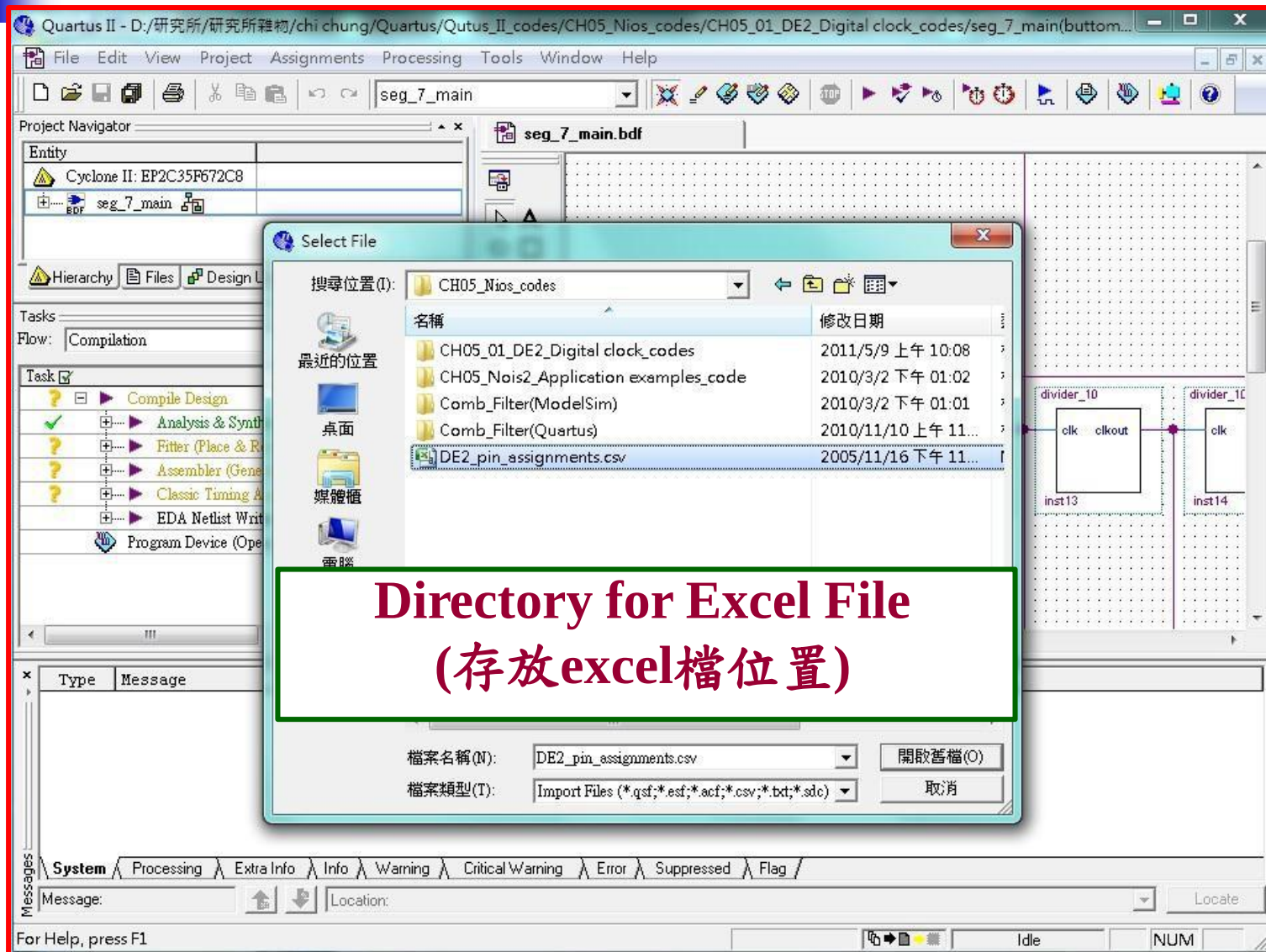
Pins Assigned with Excel File (3)



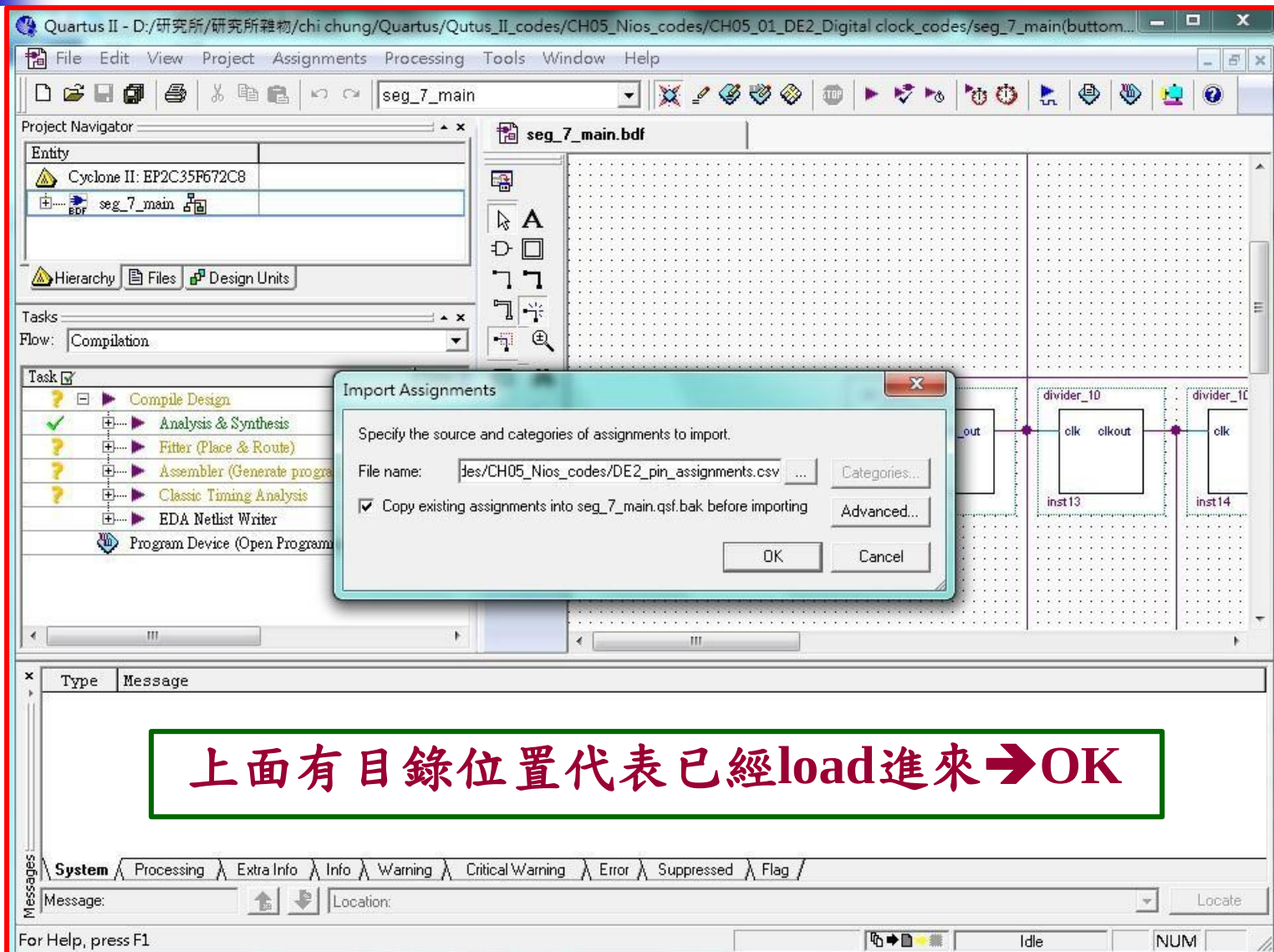
Pins Assigned with Excel File (4)



Pins Assigned with Excel File (5)

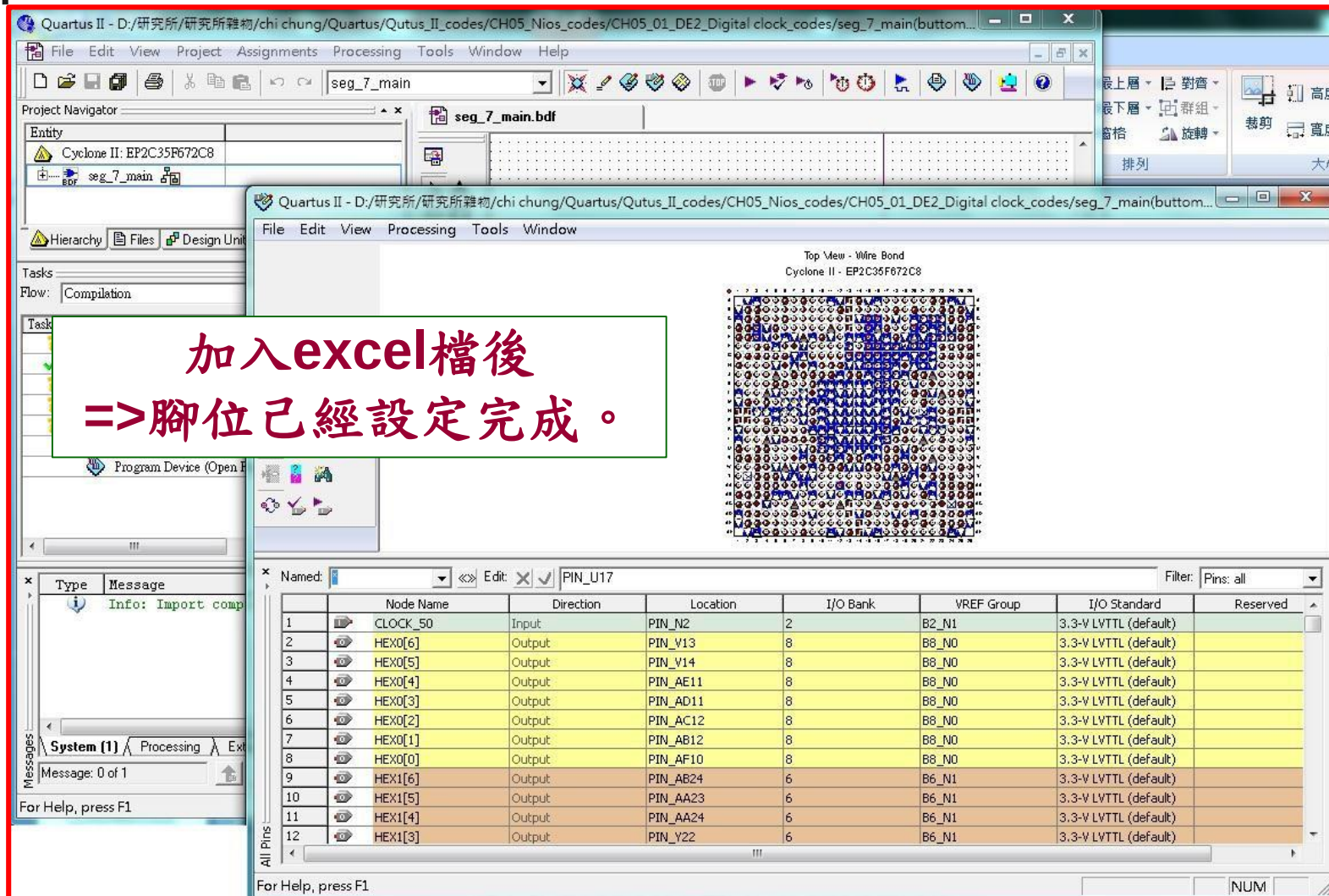


Pins Assigned with Excel File (6)



Pins Assigned with Excel File (7)

加入excel檔後
=>腳位已經設定完成。



	Node Name	Direction	Location	I/O Bank	VREF Group	I/O Standard	Reserved
1	CLOCK_50	Input	PIN_N2	2	B2_N1	3.3-V LVTTTL (default)	
2	HEX0[6]	Output	PIN_V13	8	B8_N0	3.3-V LVTTTL (default)	
3	HEX0[5]	Output	PIN_V14	8	B8_N0	3.3-V LVTTTL (default)	
4	HEX0[4]	Output	PIN_AE11	8	B8_N0	3.3-V LVTTTL (default)	
5	HEX0[3]	Output	PIN_AD11	8	B8_N0	3.3-V LVTTTL (default)	
6	HEX0[2]	Output	PIN_AC12	8	B8_N0	3.3-V LVTTTL (default)	
7	HEX0[1]	Output	PIN_AB12	8	B8_N0	3.3-V LVTTTL (default)	
8	HEX0[0]	Output	PIN_AF10	8	B8_N0	3.3-V LVTTTL (default)	
9	HEX1[6]	Output	PIN_AB24	6	B6_N1	3.3-V LVTTTL (default)	
10	HEX1[5]	Output	PIN_AA23	6	B6_N1	3.3-V LVTTTL (default)	
11	HEX1[4]	Output	PIN_AA24	6	B6_N1	3.3-V LVTTTL (default)	
12	HEX1[3]	Output	PIN_Y22	6	B6_N1	3.3-V LVTTTL (default)	

Pins Assigned with Excel File (8)

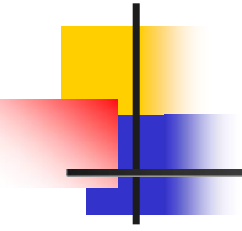
The image shows the Quartus II schematic editor interface. The main workspace displays a digital circuit design. Key components include:

- div_50m_10ms** (inst1): A divider component with inputs `clk_in` and `clk_out`.
- divider_10** (inst13, inst14): Two divider components with inputs `clk` and `clkout`.
- count60_up_down** (inst21): A counter component with inputs `clk`, `rst`, `up_down`, `hold`, and outputs `s_one[3..1]`, `s_ten[3..1]`, `m_one[3..1]`, `m_ten[3..1]`, `h_one[3..1]`, and `h_ten[3..1]`.
- led_Green** (inst18): An LED component with inputs `clk`, `rst`, `hold`, and `mode`, and output `led_out[7..0]`.
- led_Red** (inst16): An LED component with inputs `clk`, `rst`, `hold`, and `mode`, and output `led_out[17..0]`.
- divider_4_or_10** (inst3): A divider component with inputs `clk` and `clkout`.

Inputs are connected to pins: `CLOCK_50` to `PIN_N2`, `SW[14]` to `PIN_U3`, and `SW[13]` to `PIN_T7`. Outputs are connected to pins: `LEDG[7..0]` to `PIN_AE22`, `PIN_AF22`, `PIN_VV19`, `PIN_V18`, `PIN_U18`, `PIN_U17`, `PIN_AA20`, and `PIN_Y18`; and `LEDR[17..0]` to `PIN_AE23`, `PIN_AF23`, `PIN_AB21`, `PIN_AC22`, `PIN_AD22`, `PIN_AD23`, and `PIN_AD21`.

A text box on the right side of the image contains the message: **Compile後出現腳位名稱。** (Pin names appear after compilation).

The bottom status bar shows the message: **Info: Import completed. 425 assignments were written (out of 425 read). 0 non-global assignments were skipped because of entity name mismatch.**



Pins Assigned with Text File (.txt)

Pins Assigned with Text File (1)

Quartus II - D:/研究所/研究所雜物/chi chung/Quartus/Qtus_II_codes/CH05_Nios_codes/CH05_01_DE2_Digital clock_codes/seg_7_main(bottom...

File Edit View Processing Tools Window

Top View - Wire Bond
Cyclone II - EP2C35F672C8

新文字文件 (3).txt - 記事本

檔案(F) 編輯(E) 格式(O) 檢視(V) 說明(H)

```
To, Location ,  
CLOCK_50, PIN_N2  
HEX0[6], PIN_V14  
HEX0[5], PIN_V14  
HEX0[4], PIN_AE11,  
HEX0[3], PIN_AD11,  
HEX0[2], PIN_AC12,  
HEX0[1], PIN_AB12,  
HEX0[0], PIN_AF10,
```

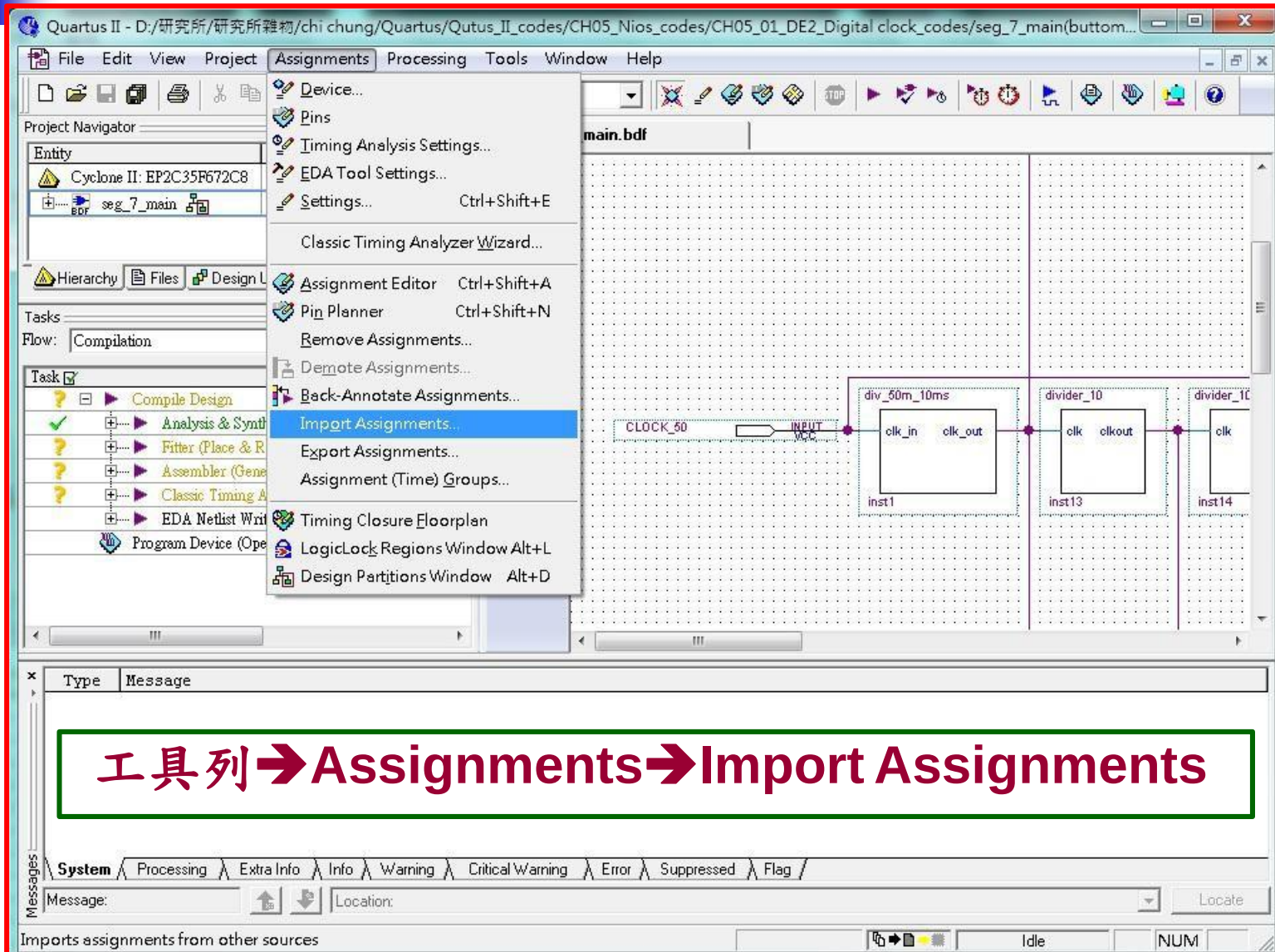
一定要用逗號隔開

To : 代表將加入的腳位名稱
Location : 代表DE2上的腳位

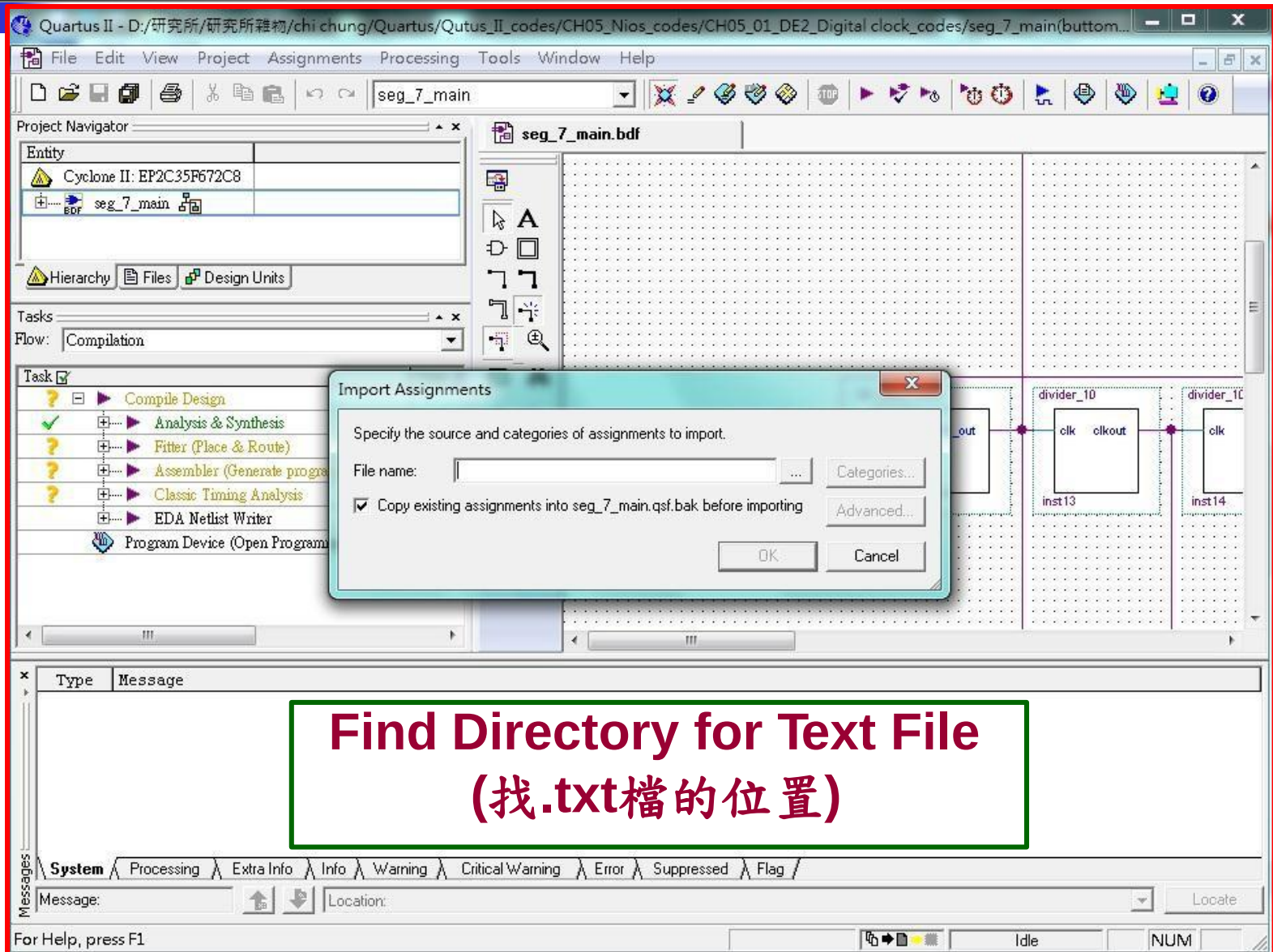
For Help, press F1

NUM

Pins Assigned with Text File (2)

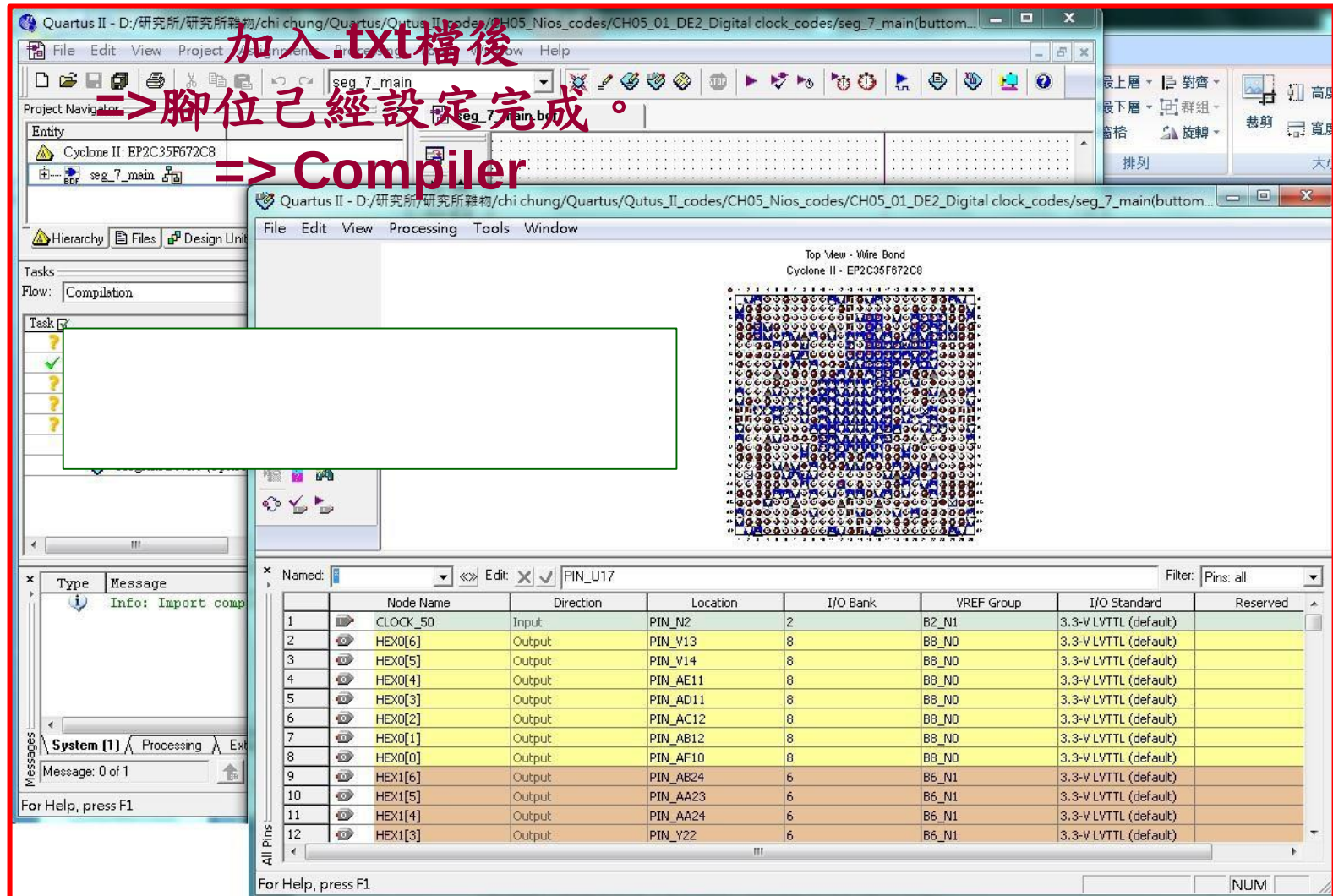


Pins Assigned with Text File (3)



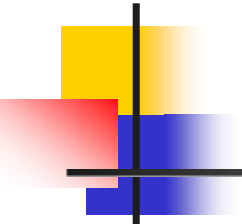
Pins Assigned with Text File (4)

加入.txt檔後
=>腳位已經設定完成。
=> Compiler



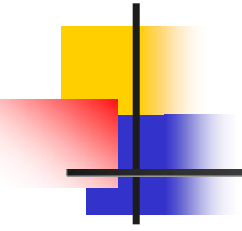
Top View - Wire Bond
Cyclone II - EP2C35F672C8

	Node Name	Direction	Location	I/O Bank	VREF Group	I/O Standard	Reserved
1	CLOCK_50	Input	PIN_N2	2	B2_N1	3.3-V LVTTTL (default)	
2	HEX0[6]	Output	PIN_V13	8	B8_N0	3.3-V LVTTTL (default)	
3	HEX0[5]	Output	PIN_V14	8	B8_N0	3.3-V LVTTTL (default)	
4	HEX0[4]	Output	PIN_AE11	8	B8_N0	3.3-V LVTTTL (default)	
5	HEX0[3]	Output	PIN_AD11	8	B8_N0	3.3-V LVTTTL (default)	
6	HEX0[2]	Output	PIN_AC12	8	B8_N0	3.3-V LVTTTL (default)	
7	HEX0[1]	Output	PIN_AB12	8	B8_N0	3.3-V LVTTTL (default)	
8	HEX0[0]	Output	PIN_AF10	8	B8_N0	3.3-V LVTTTL (default)	
9	HEX1[6]	Output	PIN_AB24	6	B6_N1	3.3-V LVTTTL (default)	
10	HEX1[5]	Output	PIN_AA23	6	B6_N1	3.3-V LVTTTL (default)	
11	HEX1[4]	Output	PIN_AA24	6	B6_N1	3.3-V LVTTTL (default)	
12	HEX1[3]	Output	PIN_Y22	6	B6_N1	3.3-V LVTTTL (default)	



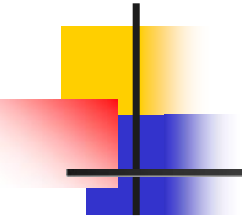
進階模組設計

- FPGA 多媒體開發平台(DE2)
- Example: Digital Clock
- IIC (I²C)



Introduction to I²C

飛利浦公司在1980年代為了讓主機板、嵌入式系統或手機得以連接低速週邊裝置，提出了一種匯流排架構，這種匯流排架構是一種兩條線(2-wire)雙向的(Bi-Directional)設計，此種匯流排架構被稱為是Inter IC的匯流排，又簡稱為 I²C 。I²C的正確讀法為 "I-squared-C"，而"I-two-C"則是另一種錯誤但被廣泛使用的讀法，在大陸地區則多以"I方C"稱之。截至2006年11月1日為止，**使用I²C協定不需要為其專利付費，但製造商仍然需要付費以獲得I²C從屬裝置位址。**

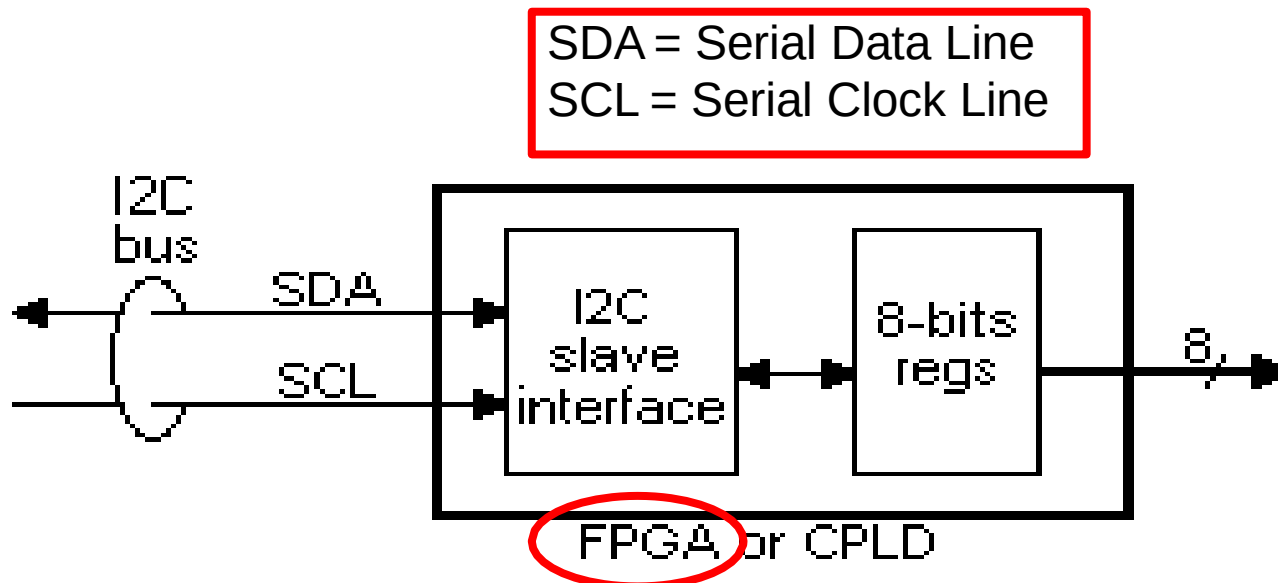


Features of I2C (1)

1. 只有兩條導線的設計，分別為串列資料線(SDA)及串列時鐘訊號線(SCL)這對IC來講是有很大的吸引力，越少的PIN-Count代表的是越簡單的PCB設計和更低的包裝(Package)成本。
2. 兩條導線同時連接到bus上所有的master-slave；惟在同一時刻裡，只會有一組master-slave在操作，如果有一個以上的master-slave要啟動bus，I2C具有仲裁的機制可以避免碰撞或破壞資料。
3. I²C的參考設計使用一個7位元長度的位址空間但保留了16個位址，所以在一組匯流排最多可和112($=2^7-16$)個節點通訊。
4. 常見的I²C匯流排依傳輸速率的不同而有不同的模式：標準模式（100 Kbit/s）、低速模式（10 Kbit/s），但時脈頻率可被允許下降至零，這代表可以暫停通訊。
5. 新一代的I²C匯流排可以和更多的節點（支援10位元長度的位址空間）以更快的速率通訊：快速模式（400 Kbit/s）、高速模式（3.4 Mbit/s）。

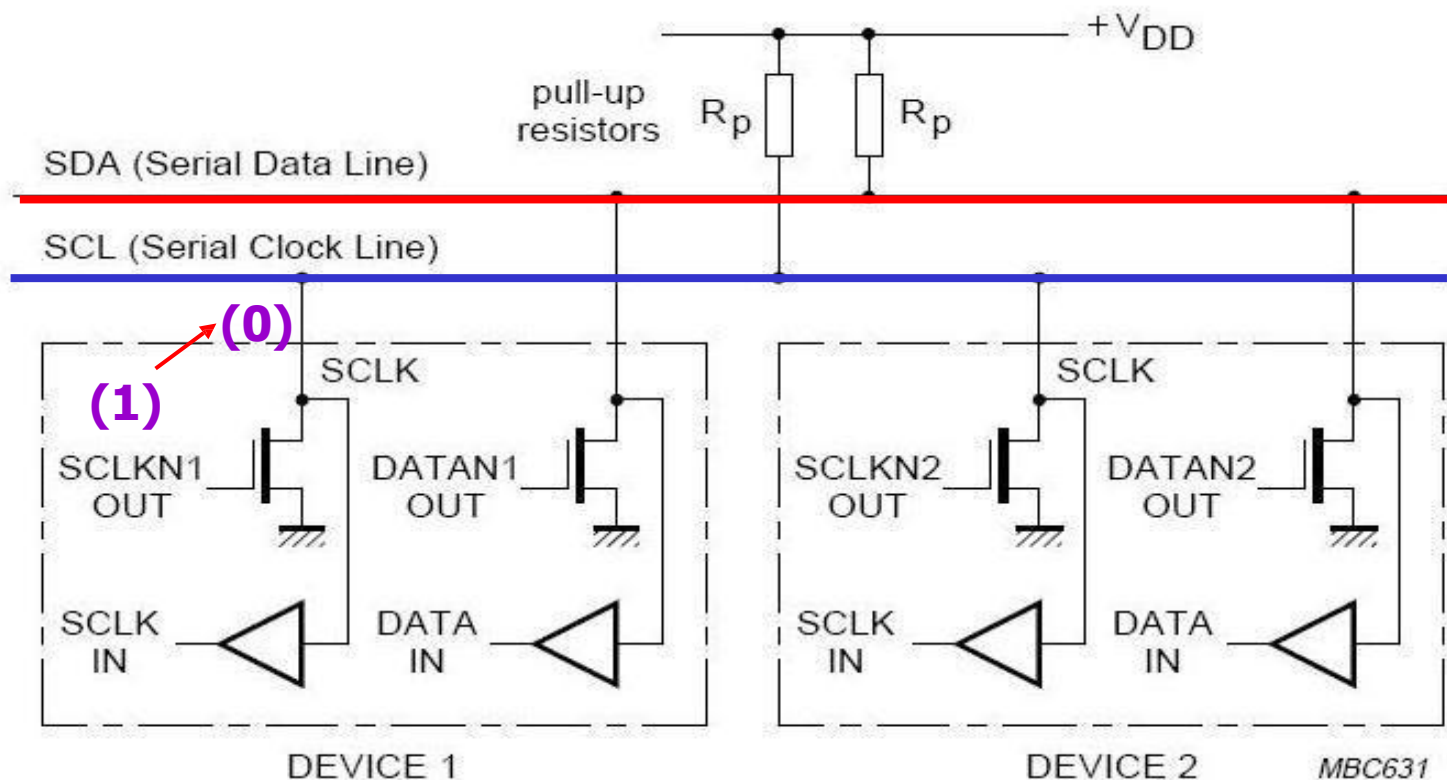
Features of I2C (2)

6. 採用I2C的原因：節省成本和設計上的簡易性；而非極高的頻寬，如果ASIC需要對外部記憶體作經常性的存取，就不適合採用I2C當介面。
7. 定址和資料傳輸的協定允許全部由軟體來控制。
8. 在IC和IC之間不需要設計介面電路。
9. 極低的電流消耗量與寬廣的操作溫度。



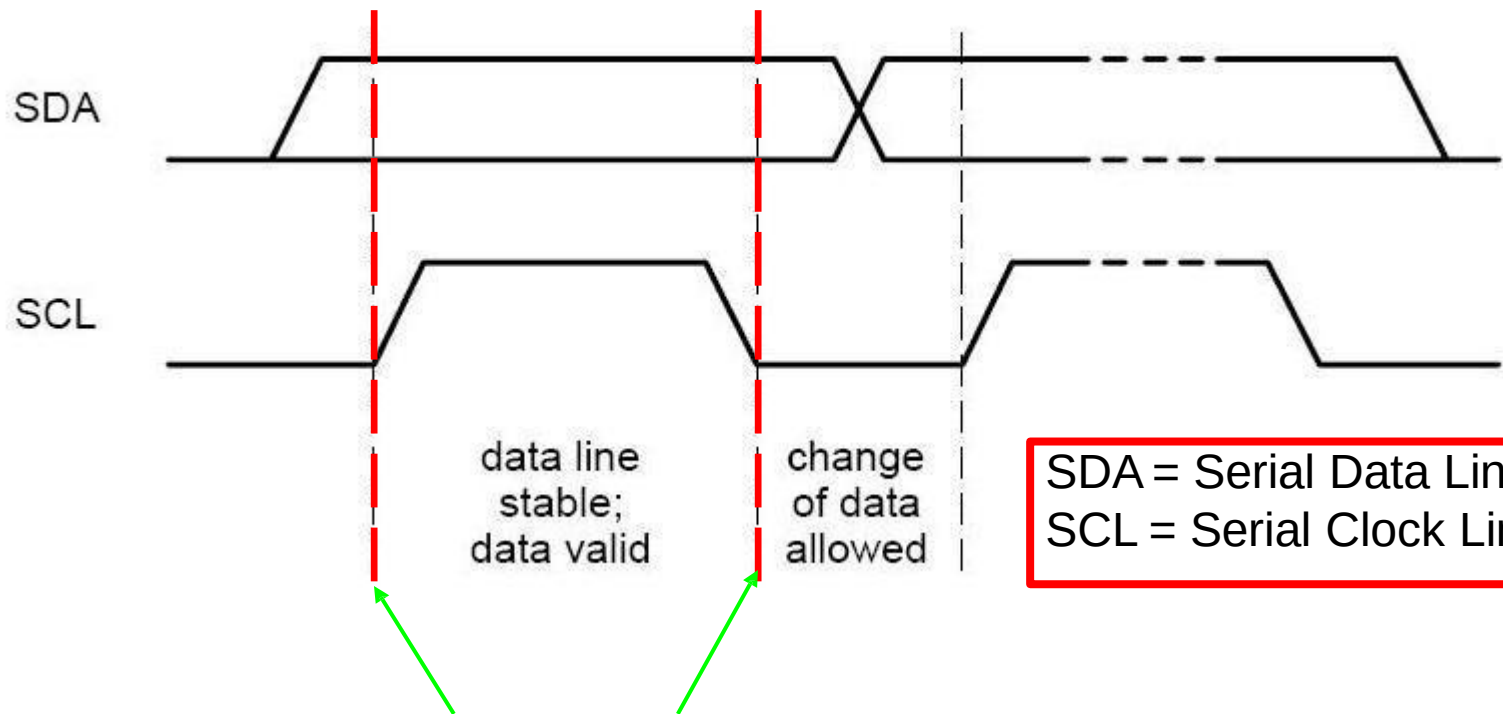
Open-Drain Bus Architecture

I2C容許有許多個元件的Drain(汲極)連接到匯流排上，因此只要匯流排有元件將訊號Pull Low (SCLK=1 or DATA=1)，整個匯流排上面的元件都會看到低準位(0)的訊號，這也是匯流排仲裁的機制：如果匯流排上沒有動作，則兩條訊號會一直保持在High (1)。



Bit Transfer of I2C

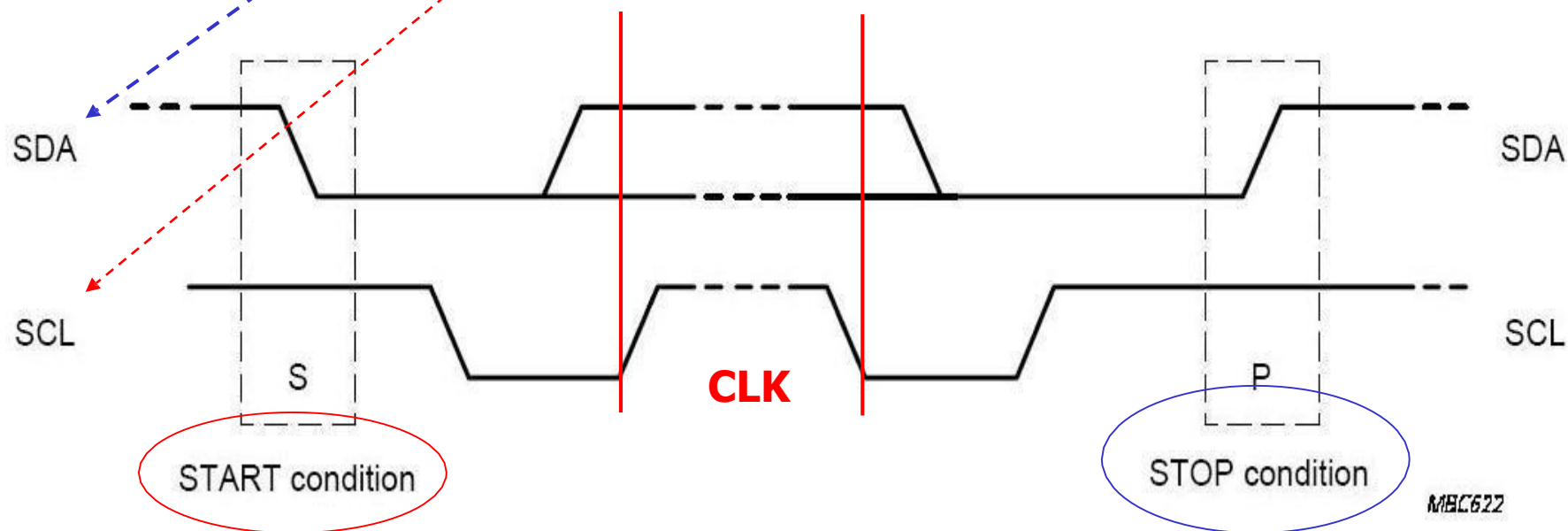
不論傳輸的是位址(Address)、資料(Data)或控制信號(Signal)，I2C均要求傳輸的資料必須在**時脈訊號前後達到穩定**，因此I2C並不能算是同步傳輸，不過這種傳輸方式卻能有很好的**雜訊免疫能力**。



CLK前後達到穩定

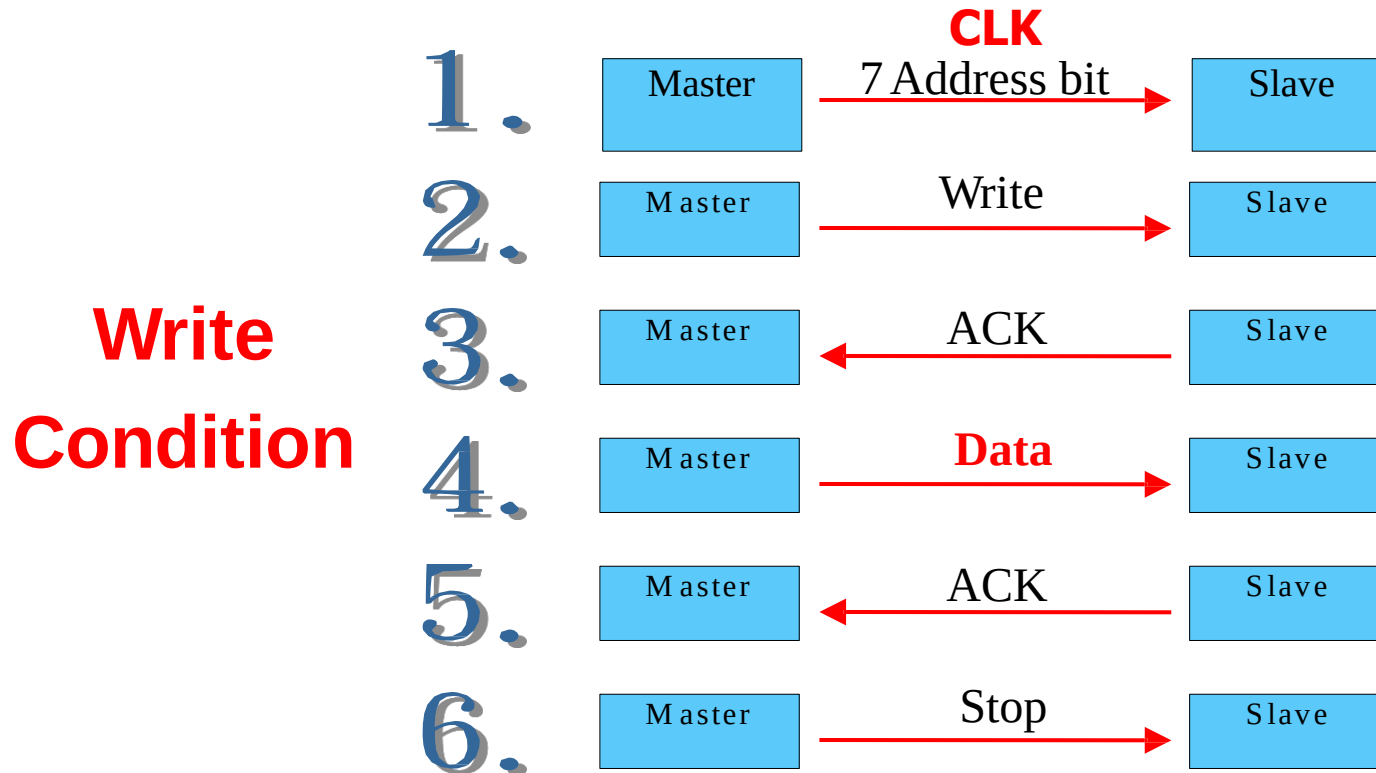
Start-Stop condition

I2C的協定信號相當簡易，僅由SCL、SDA、Pull Low、Pull High的先後順序即可決定協定；其中，**SCL (clock)**一定由**Master(即啟動Bus傳輸的元件)**發出，而**SDA (data)**則不一定，要視傳遞的是位址或資料或者是控制訊號而定。



Data Write Condition

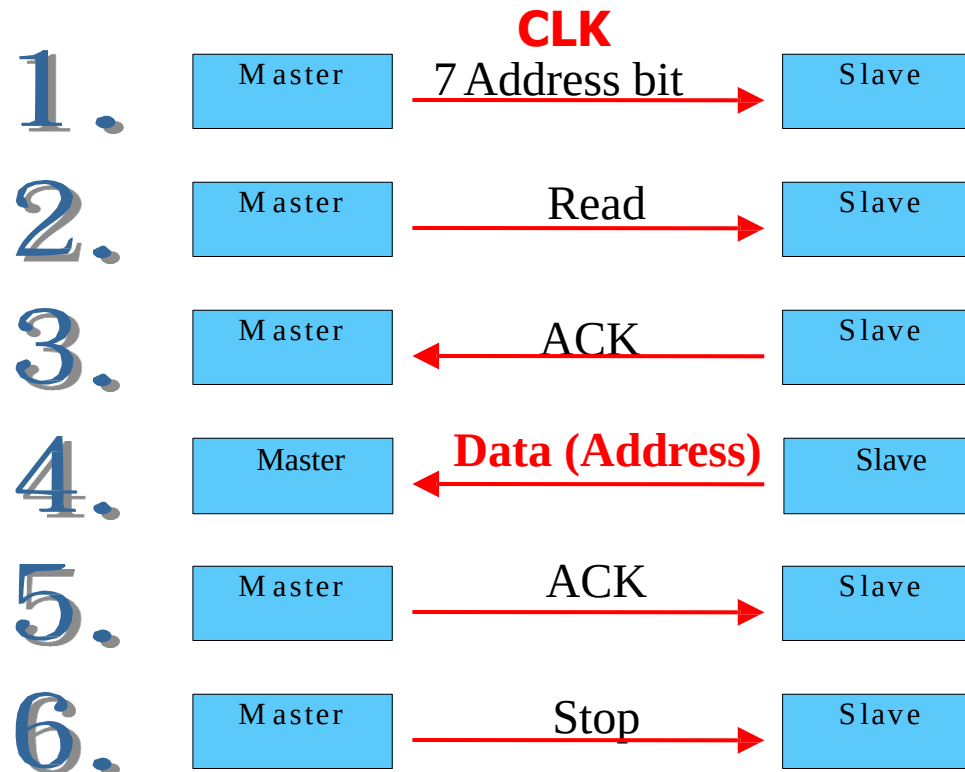
所有的傳輸可分為讀取和寫入，而不論是傳輸或者是寫入的動作，**Master**會先發出7個**Address bit**，這7個**Address bit**的順序是**MSB**先，最後才是**LSB**。然後再發出Write (Active Low)或Read (Active High)的信號，接著Slave必須回應一個**ACK**的信號，如果是Write的話，Master就把欲傳輸的資料傳給Slave，然後由Slave回應一個**ACK**信號，最後由Master發出結束信號；



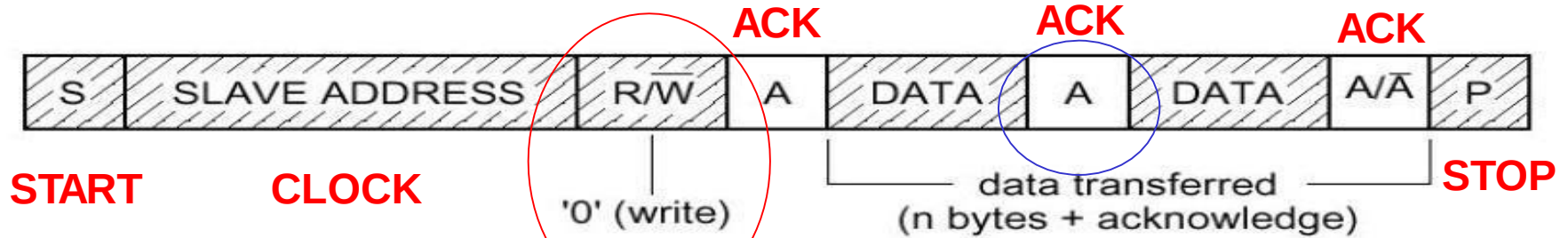
Data Read Condition

如果是Read的話，就由**Slave**回應**Master**相對位址的資料，然後由Master回應一個ACK，最後由Master發出結束信號。如果Slave回應的是Not ACK，Master可以選擇結束信號，或者是再傳一次。

Read Condition



Simplified Data Read/Write Format



 from master to slave

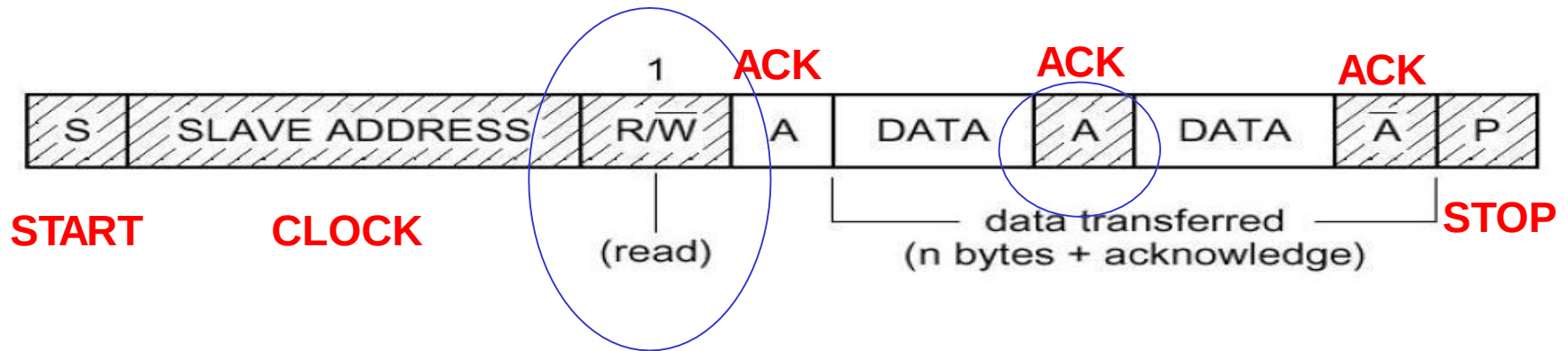
 from slave to master

A = acknowledge (SDA LOW)

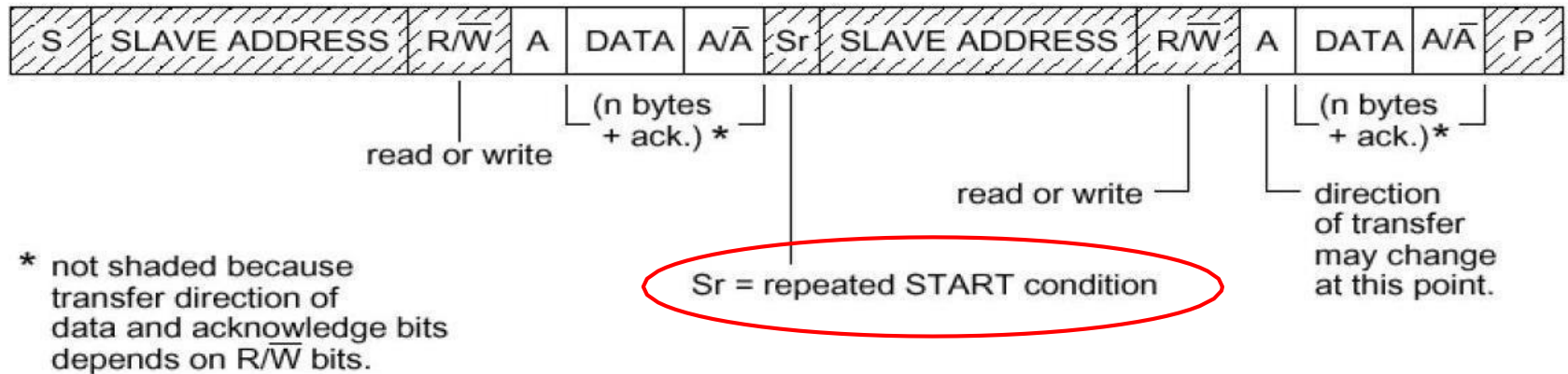
$\bar{A}$ = not acknowledge (SDA HIGH)

S = START condition

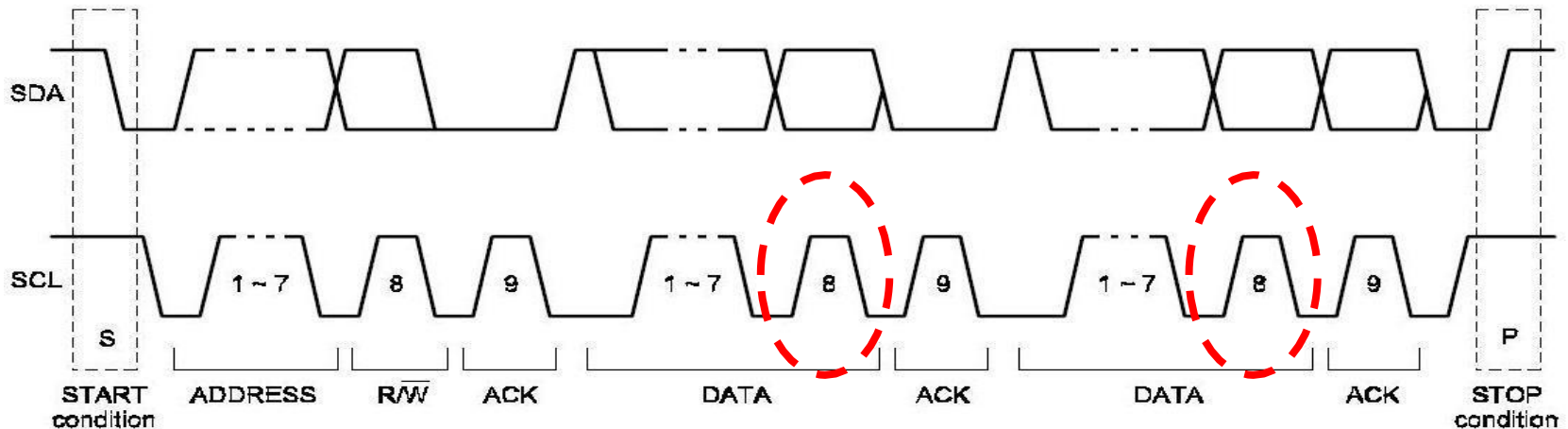
P = STOP condition



A Completed Data R/W Format and Data Transfer



Serial Data = SDA A Completed Data R/W Format

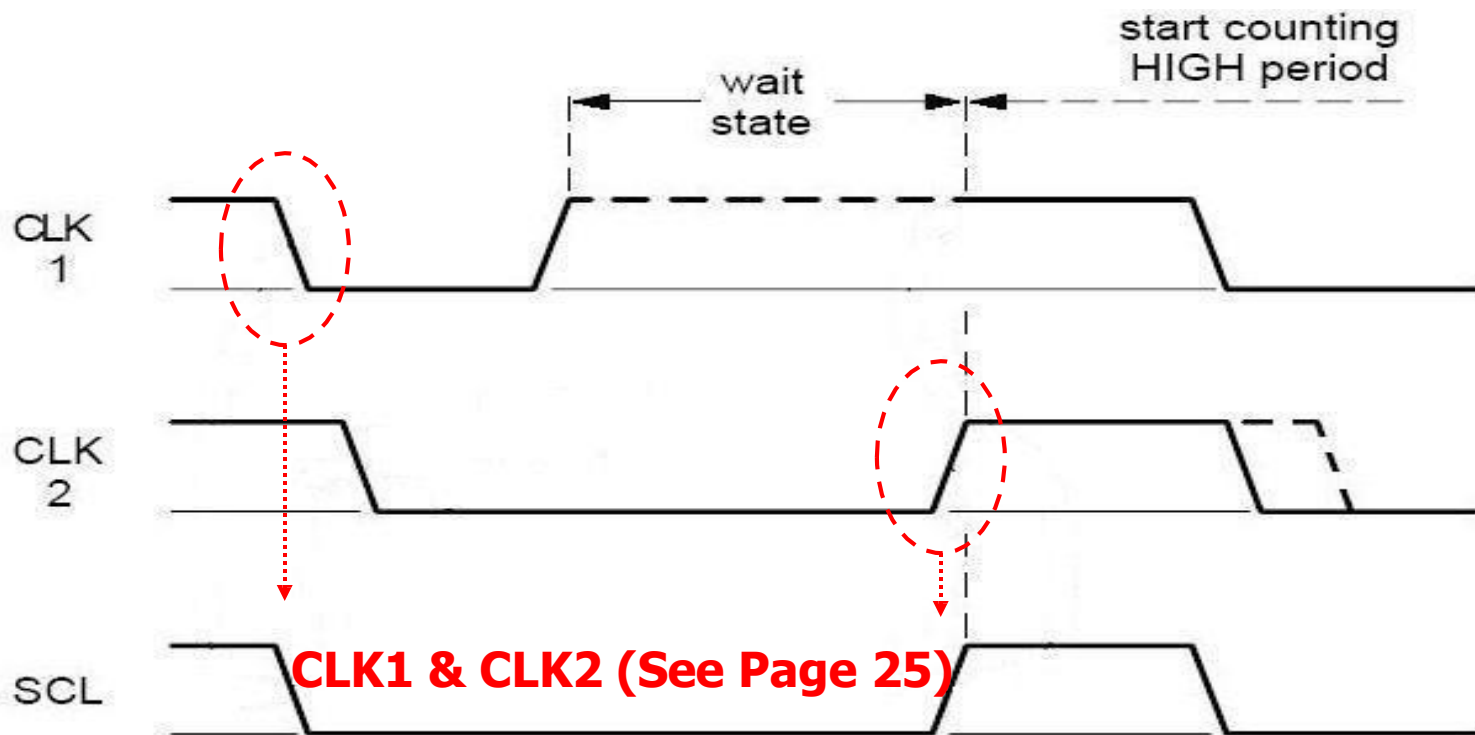


Serial Clock = SCL

A Completed Data Transfer

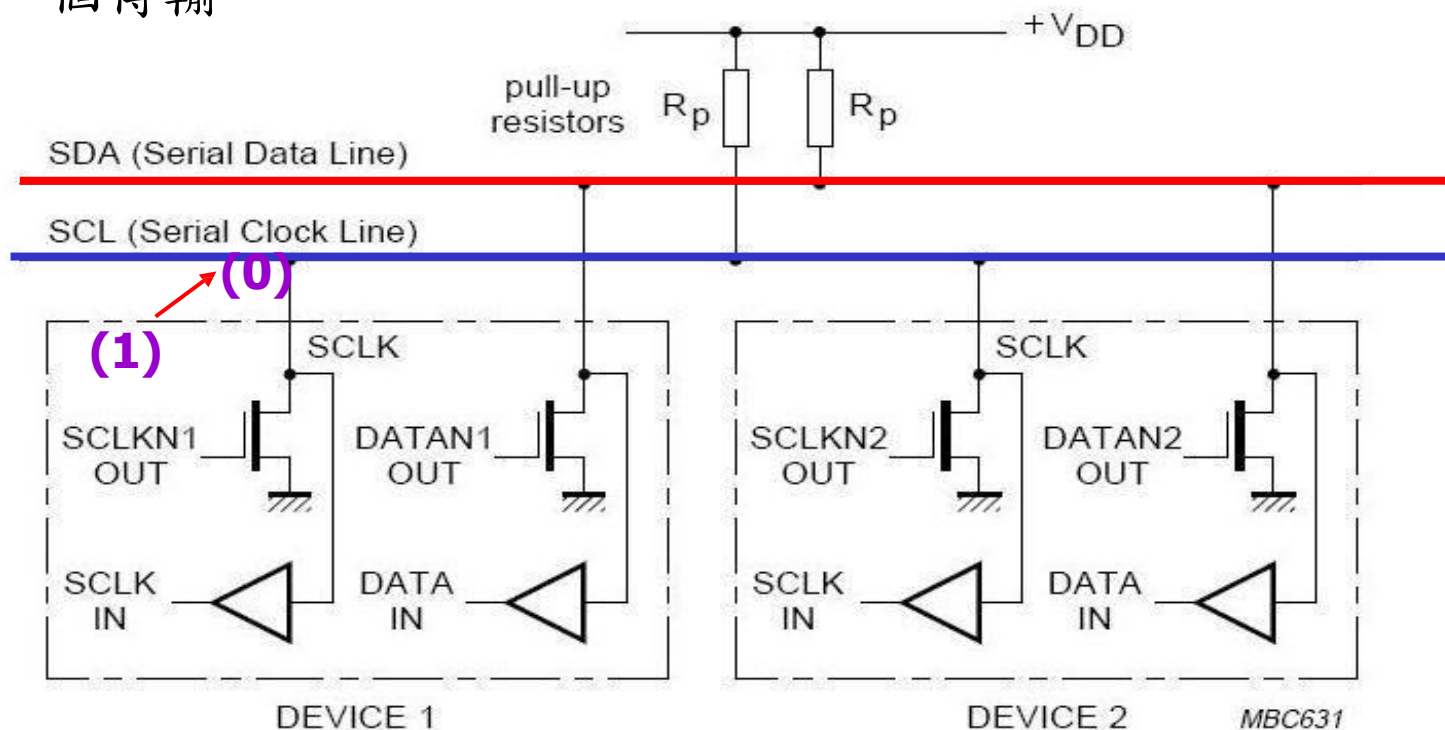
Clock Synchronous

當有超過一個以上的Master欲啟動傳輸時，將會有兩個以上的CLK Source
這兩個CLK Source到底哪一個才是正確的呢？ 真正Bus上看到的clock信號
可視為兩個 CLK Source做AND運算。

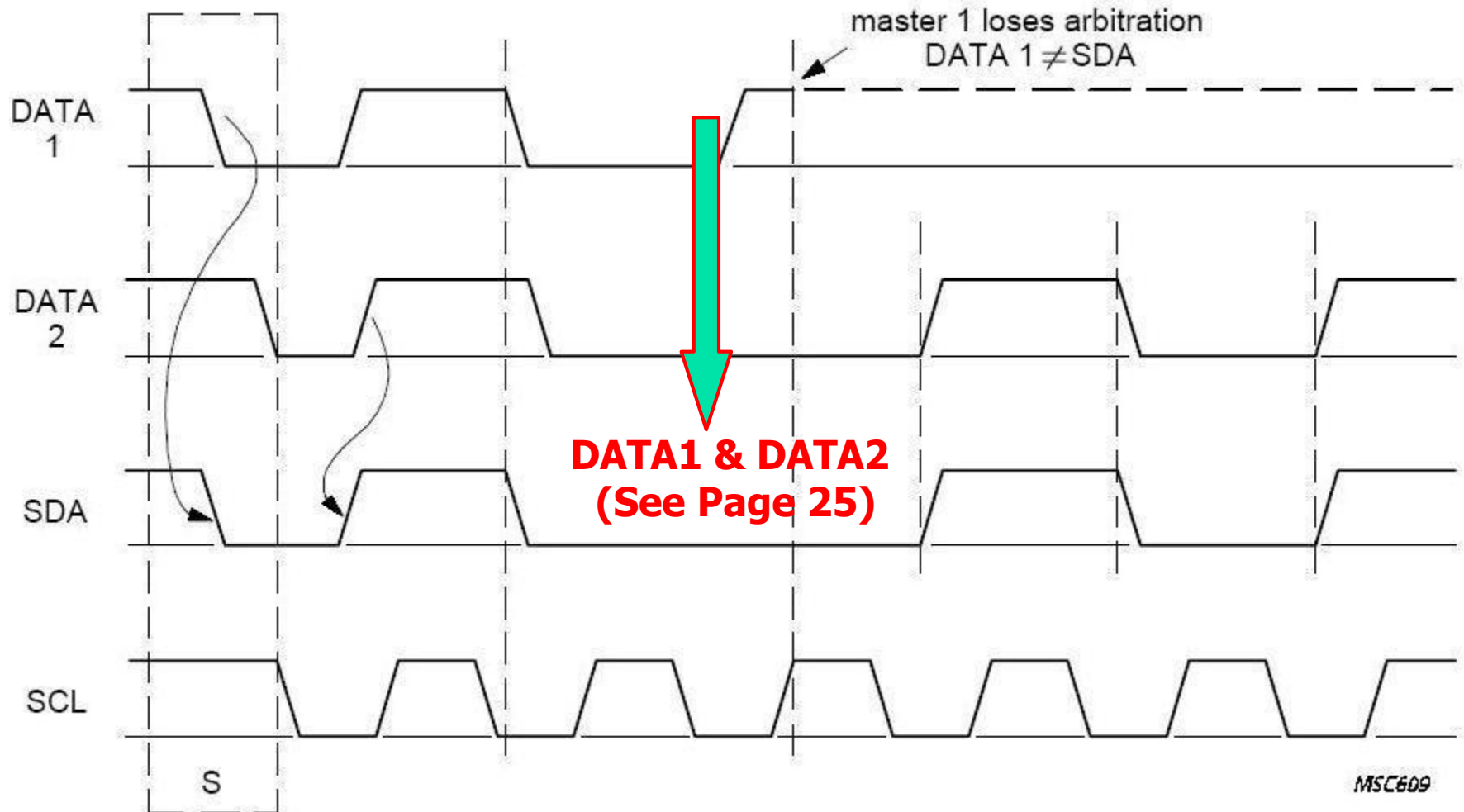


Arbitration (仲裁)

I2C為了求取最簡易經濟的設計，並沒有一個仲裁者來決定匯流排的使用歸屬權，而是同樣由Open-Drain的架構，誰最先將位址線Pull Low就先擁有匯流排的使用權；如果匯流排上有傳輸在進行，其他欲啟動傳輸的元件必須偵測到STOP Condition，才能啟動另一個傳輸。



Example for Arbitration



I2C Circuit Module (1)

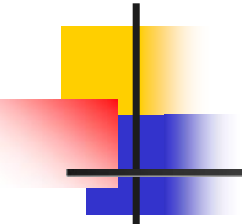
Declare (Initial)

6_01_I2C

```
module I2C (CLOCK, I2C_SCL, I2C_SDA, I2C_DATA, W_R, END, ACK, RESET,  
            SD_COUNTER);
```

```
input CLOCK;  
input [14:0]I2C_DATA;           //DATA: (SLAVE_ADDR[6..0],DATA[7..0])  
input W_R;                     //Write or Read  
input RESET;  
inout I2C_SDA;               //I2C DATA  
output I2C_SCL;                //I2C CLOCK  
output END;  
output ACK;  
output [4:0] SD_COUNTER;       //Counter
```

```
reg I2C_SDA;  
reg I2C_SCL;  
reg END;  
reg ACK;  
reg [14:0]SD;                  //SD = I2C_DATA (先暫存在SD，再送出)  
reg [4:0]SD_COUNTER;
```

I2C Circuit Module (2)

Counter Part

```
always @(negedge RESET or posedge CLOCK )
```

```
begin
```

```
if (!RESET)
```

若 RESET =0 ,SD_COUNTER 清空

```
    SD_COUNTER=5'b0;
```

```
else
```

```
    begin
```

```
        if (SD_COUNTER < 5'b10110)
```

若SD_COUNTER < 22(動作數), 清空

```
            SD_COUNTER=SD_COUNTER+1;
```

否則, 繼續計數

```
        else
```

```
            SD_COUNTER=0;
```

```
    end
```

```
end
```

I2C Circuit Module (3)

Main Code

```
always @(negedge RESET or posedge CLOCK)
begin
```

```
  if (!RESET)
```

```
    begin
```

```
      I2C_SCL=1;I2C_SDA=1; ACK=0; END=1; //Reset action
```

```
    end
```

```
  else
```

```
    case (SD_COUNTER)
```

```
      5'd0 : begin ACK=0 ; END=0; I2C_SDA=1; I2C_SCL=1;end
```

```
      5'd1 : begin ACK=1 ;SD=I2C_DATA; I2C_SDA=0;end
```

```
      5'd2 : begin ACK=0 ;I2C_SCL=0;end
```

```
      5'd3 : I2C_SDA=SD[14];
```

```
      5'd4 : I2C_SDA=SD[13];
```

```
      5'd5 : I2C_SDA=SD[12];
```

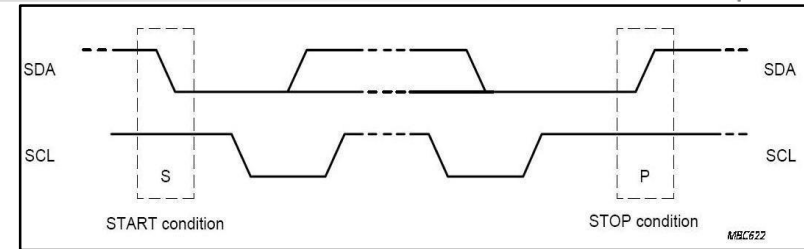
```
      5'd6 : I2C_SDA=SD[11];
```

```
      5'd7 : I2C_SDA=SD[10];
```

```
      5'd8 : I2C_SDA=SD[9];
```

```
      5'd9 : I2C_SDA=SD[8];
```

```
      5'd10 : begin I2C_SDA=W_R; ACK=1; end 再傳送W/R信號，接著給一個ACK
```



H → L

將I2C_DATA的輸入送到I2C_SDA

Start Condition

先傳送7-bit Address

//7bit SLAVE_ADDR (MSB)

·
·
·
·
·

//7bit SLAVE_ADDR (LSB)

I2C Circuit Module (4)

```
5'd11 : begin I2C_SDA=SD[7]; ACK=0; end //8bit DATA (MSB)
5'd12 : I2C_SDA=SD[6];
5'd13 : I2C_SDA=SD[5];
5'd14 : I2C_SDA=SD[4];
5'd15 : I2C_SDA=SD[3];
5'd16 : I2C_SDA=SD[2];
5'd17 : I2C_SDA=SD[1];
5'd18 : I2C_SDA=SD[0];
5'd19 : ACK=1; //8bit DATA (LSB)
```

資料傳送結束後，最後再給一個ACK

Data Transfer

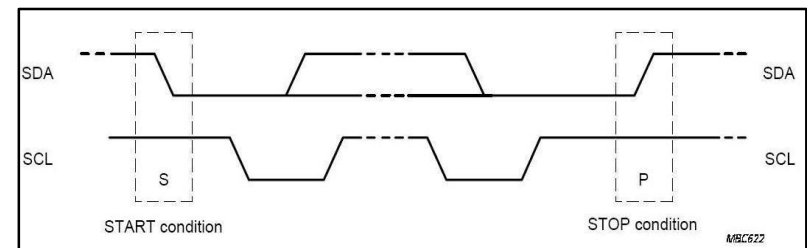
```
5'd20 : begin I2C_SDA=1'b0; I2C_SCL=1'b0; ACK=0; end
5'd21 : I2C_SCL=1'b1;
5'd22 : begin I2C_SDA=1'b1; END=1; end
```

Stop Condition

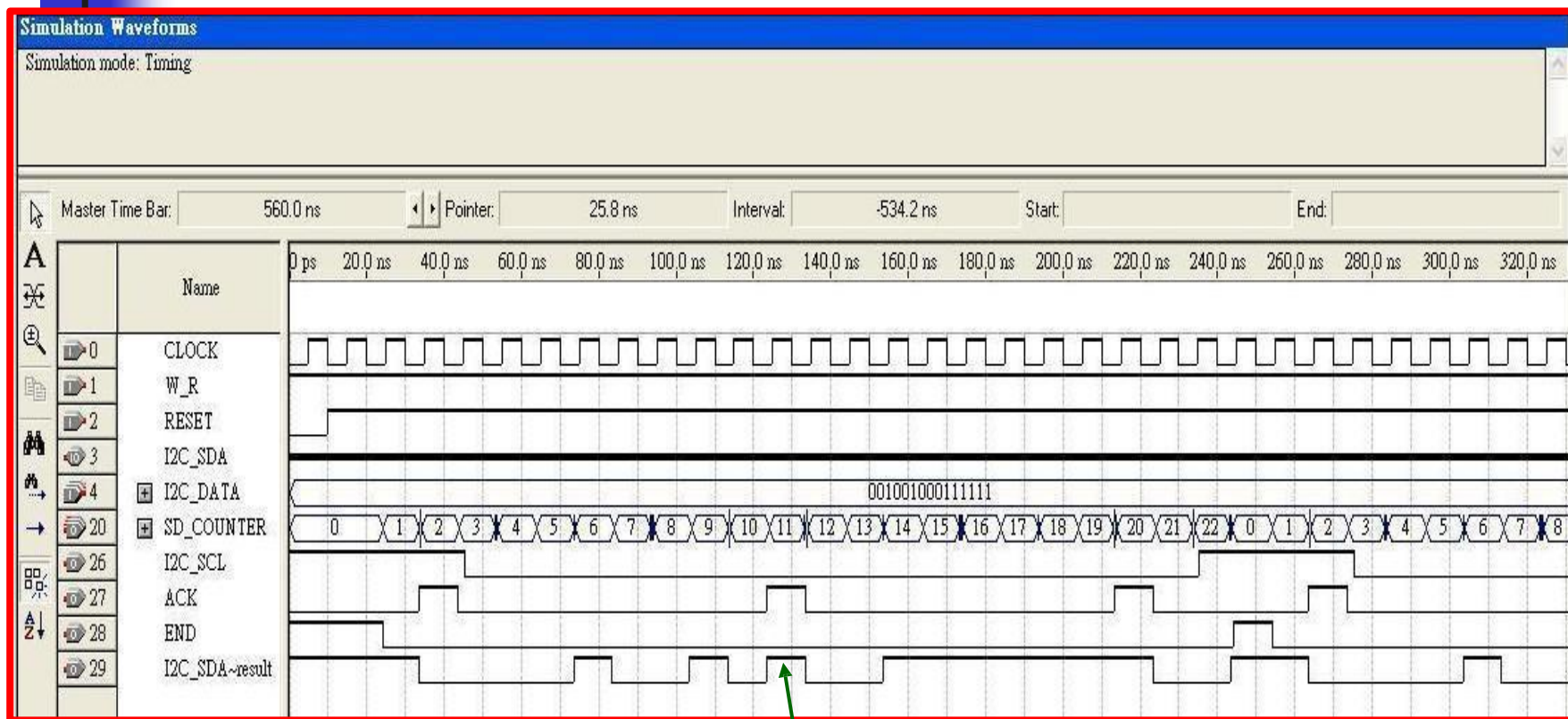
最後結束時給一個END信號

```
endcase
end
endmodule
```

L → H



I2C Circuit Module (5)



0010010,ACK,00111111

Write = 0
Read = 1

